集群计算机与科学计算

张武生、薛巍
清华大学计算机系



任课教师介绍

	张武生	薛　巍
单位	计算机系高性能计算技术研究所	
电话	15810786106	13910010177
邮件地址	zws@tsinghua.edu.cn	xuewei@tsinghua.edu.cn
研究方向	集群计算	科学计算、科学智能

欢迎各位同学随时交流讨论☺

课程内容：

- 集群计算机介绍?
- 集群计算机能解决什么问题?
- 如何在集群计算机上编程序?
- 无所不在的并行处理
- 并行计算的未来挑战

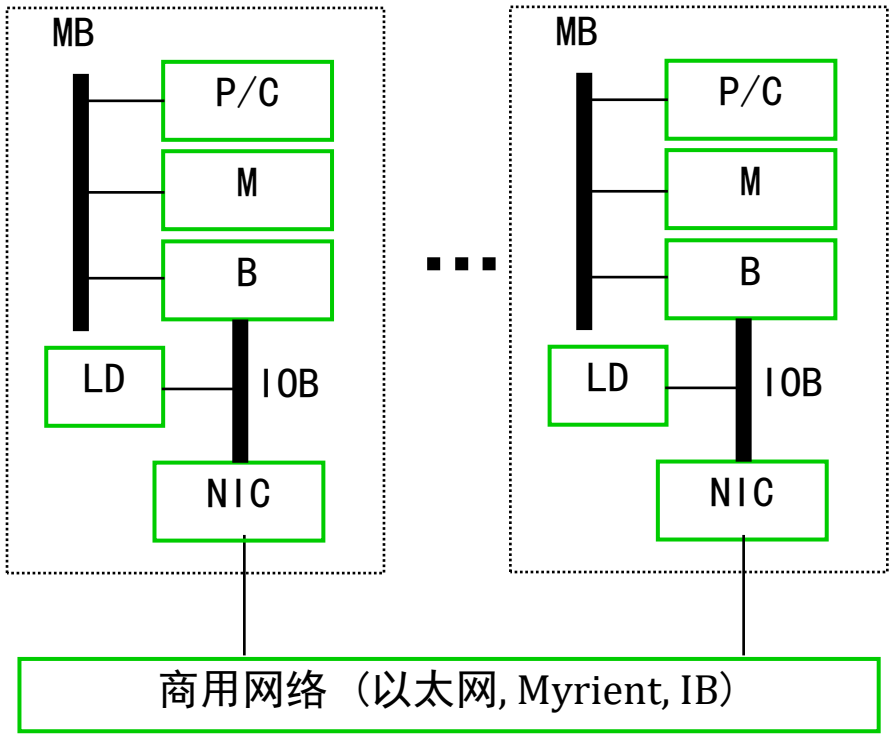
课程内容：

- 集群计算机介绍?
- 集群计算机能解决什么问题?
- 如何在集群计算机上编程序?
- 无所不在的并行处理
- 并行计算的未来挑战

商用计算系统构建集群计算机



集群计算机



B：存储总线与I/O总线的接口；LD：本地磁盘
IOB：I/O总线

完全采用
商用硬件和软件

- **每个结点都是一个完整计算机**
- **每个结点上驻留有完整的操作系统**
- **各结点通过低成本的商用网络互连**

集群计算的起源



CDC6600, 1964-1969
A top speed of 40 MHz, 3 megaFLOPS



Cray-1, 1976
Integrated circuits used, 136 megaFLOPS



1993-1996, Numerical Wind Tunnel
600 gigaFLOPS



1996, ASCI Red
First to 1 teraFLOPS

The final system had a total of 9298 Pentium II OverDrive processors, each clocked at 333 MHz. The system consisted of 104 cabinets, taking up about 2500 square feet (230 m²). The system was designed to use commodity mass-market components and to be very scalable.

集群计算的起源



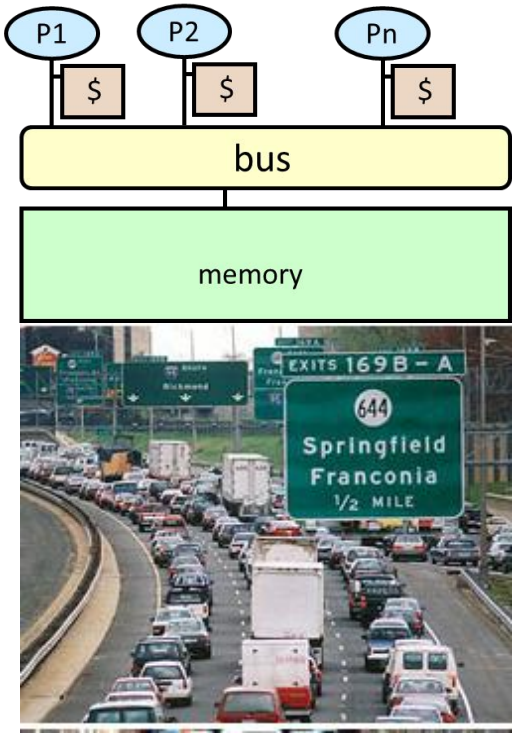
- **1970s-1980s, Burroughs large systems 5700, ARC, VMScluster**

- **An experiment in parallel computing systems (1994)**
- **Established vision of low cost, high end computing**
  - **Cost effective because it uses off-the-shelf parts**
- **Demonstrated effectiveness of PC clusters for some (not all) classes of applications**
- Provided networking software
- Conveyed findings to broad community
- Tutorials and book
- Design standard to rally community!
- Standards beget:
  books, trained people,
  software … virtuous cycle

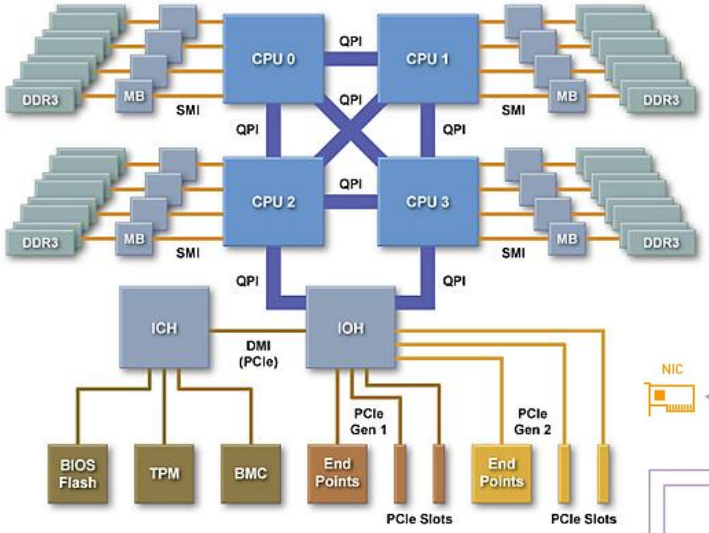


Adapted from Gordon Bell, presentation at Salishan 2000

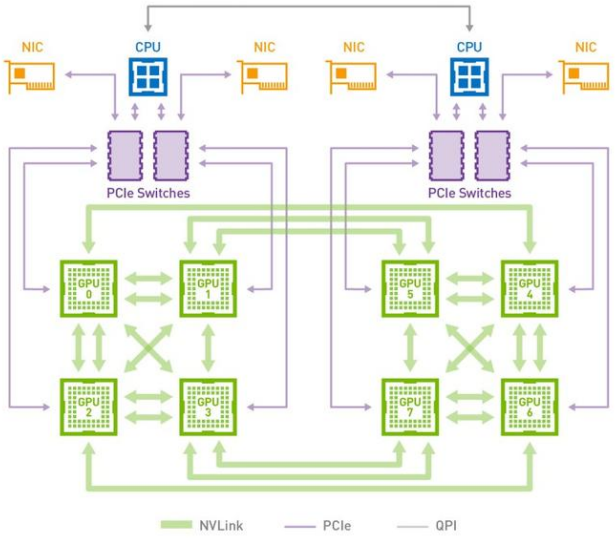
集群计算机的节点可以很复杂



Uniform Memory Access (UMA)



Non-Uniform Memory Access (NUMA)

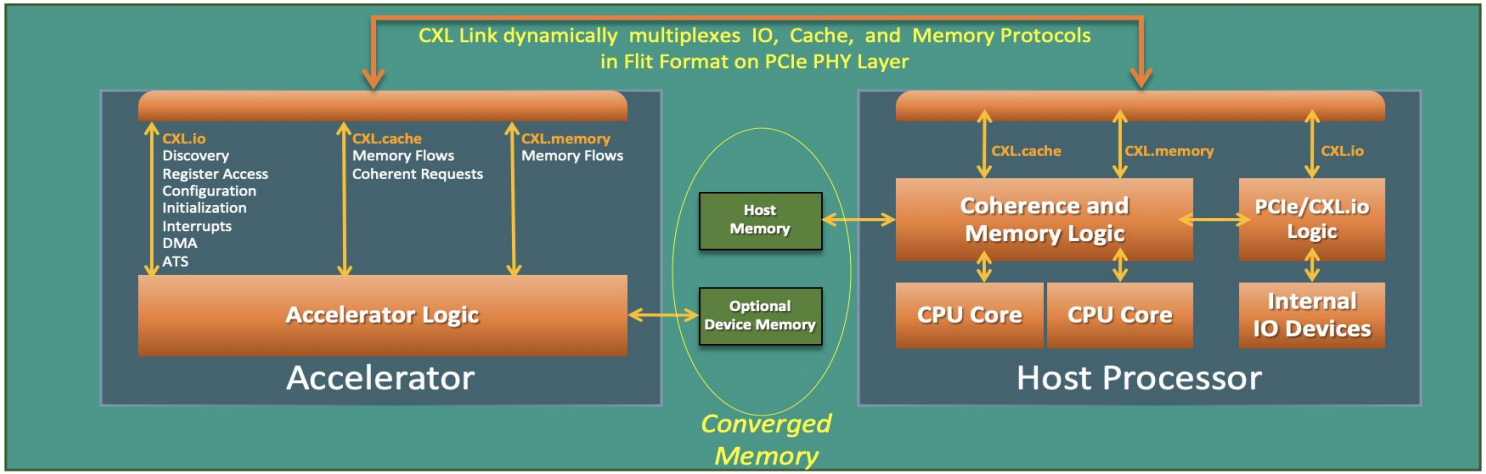


NVLink

集群计算机的节点可以很复杂



CXL (Compute Express Link) 技术是一种新型的高速互联技术，旨在提供更高的数据吞吐量和更低的延迟，以满足现代计算和存储系统的需求。它最初由英特尔、AMD等联合推出，并得到了包括谷歌、微软等公司在内的大量支持。
CXL的目标：解决CPU和设备、设备和设备之间的内存鸿沟。服务器有巨大的内存池和数量庞大的基于PCIe运算加速器，每个上面都有很大的内存。内存的分割已经造成巨大的浪费、不便和性能下降。CXL就是为解决这个问题而诞生。



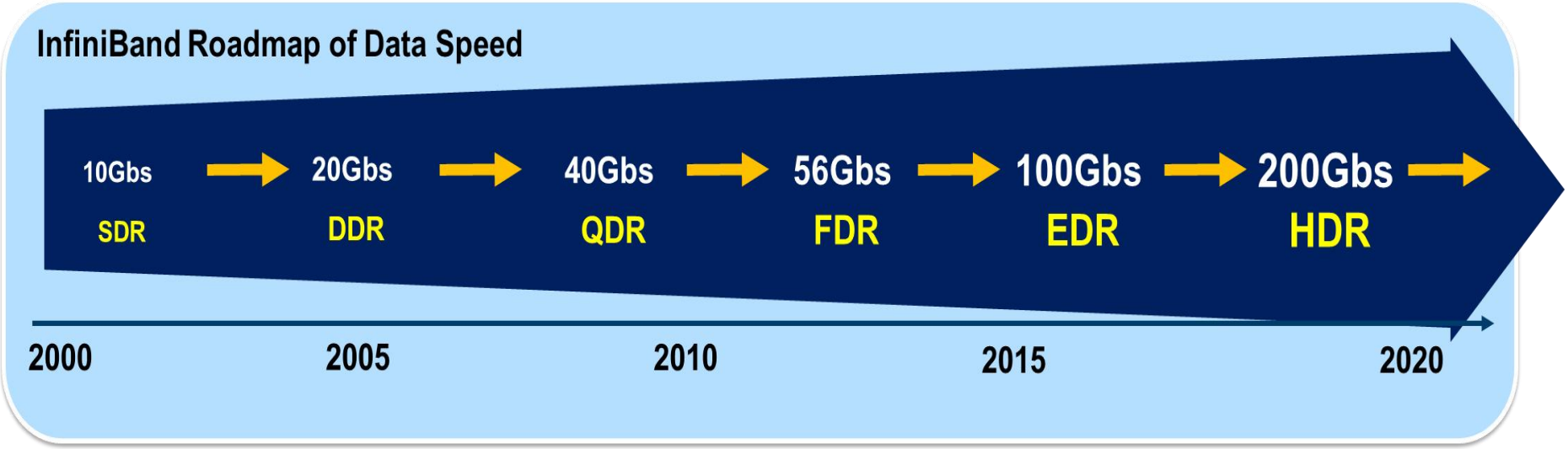
什么是CXL（Compute Express Link）技术？一文读懂CXL - 知乎

集群计算机的网络也可以很给力

- SDR - Single Data Rate
- DDR - Double Data Rate
- QDR - Quad Data Rate
- FDR - Fourteen Data Rate

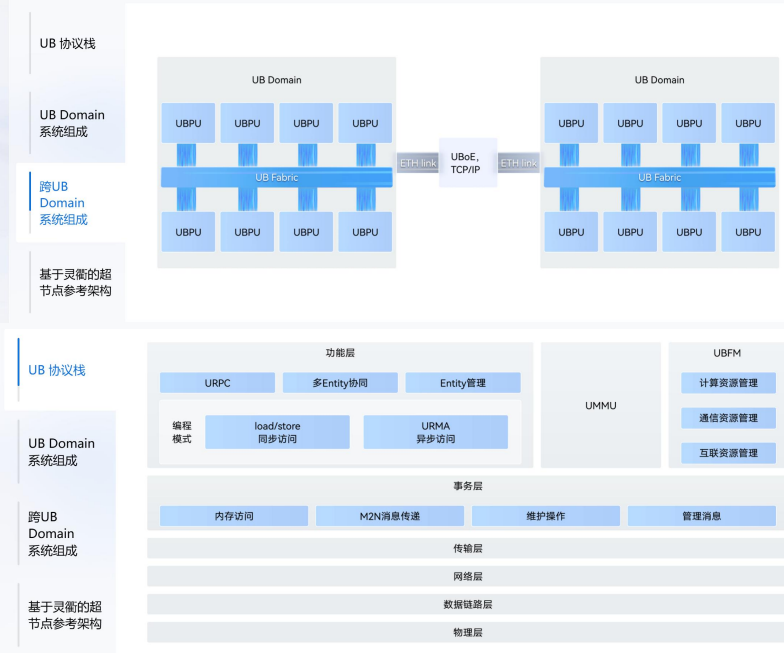
- EDR - Enhanced Data Rate
- HDR - High Data Rate
- NDR - Next Data Rate





全局内存访问与超节点构建：华为灵衢



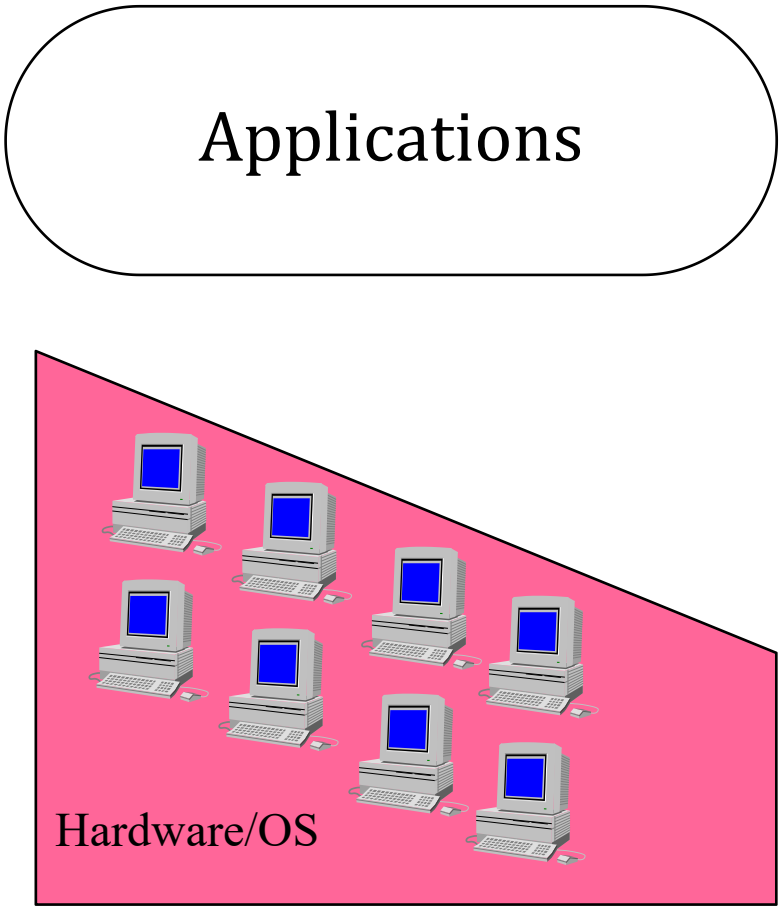






[灵衢社区]

集群计算机的核心在软件

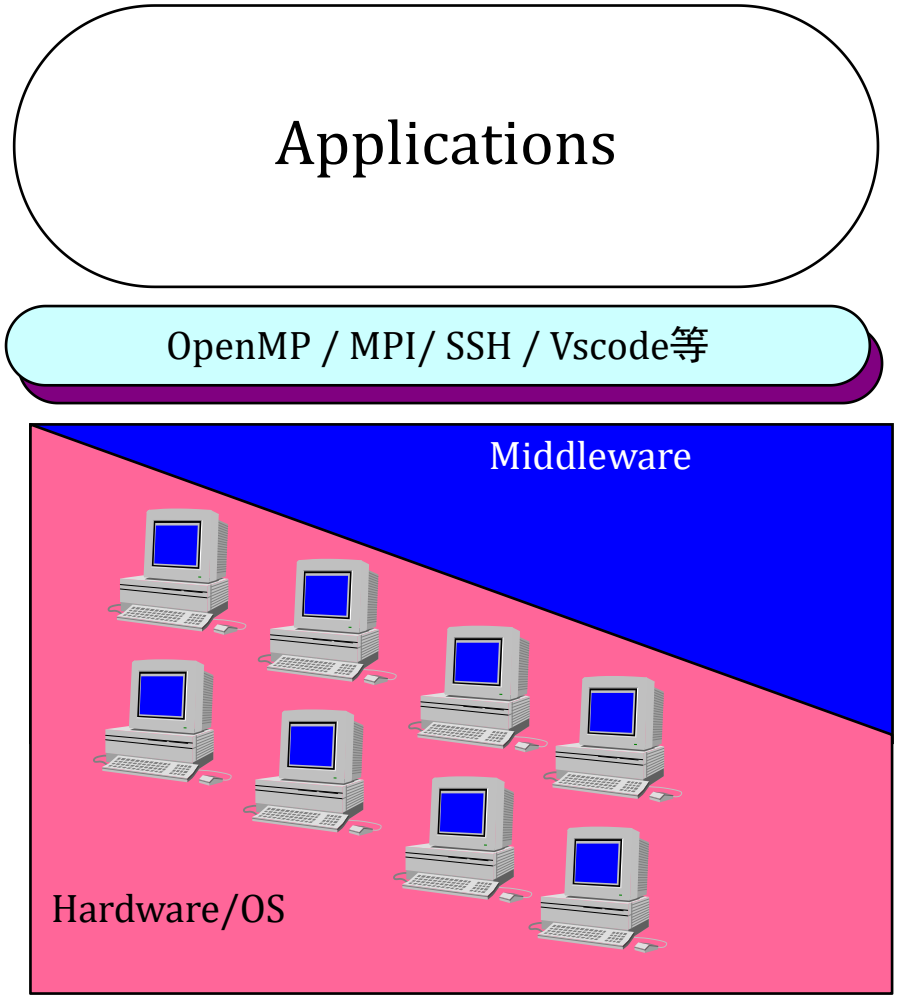


1 如何安装？

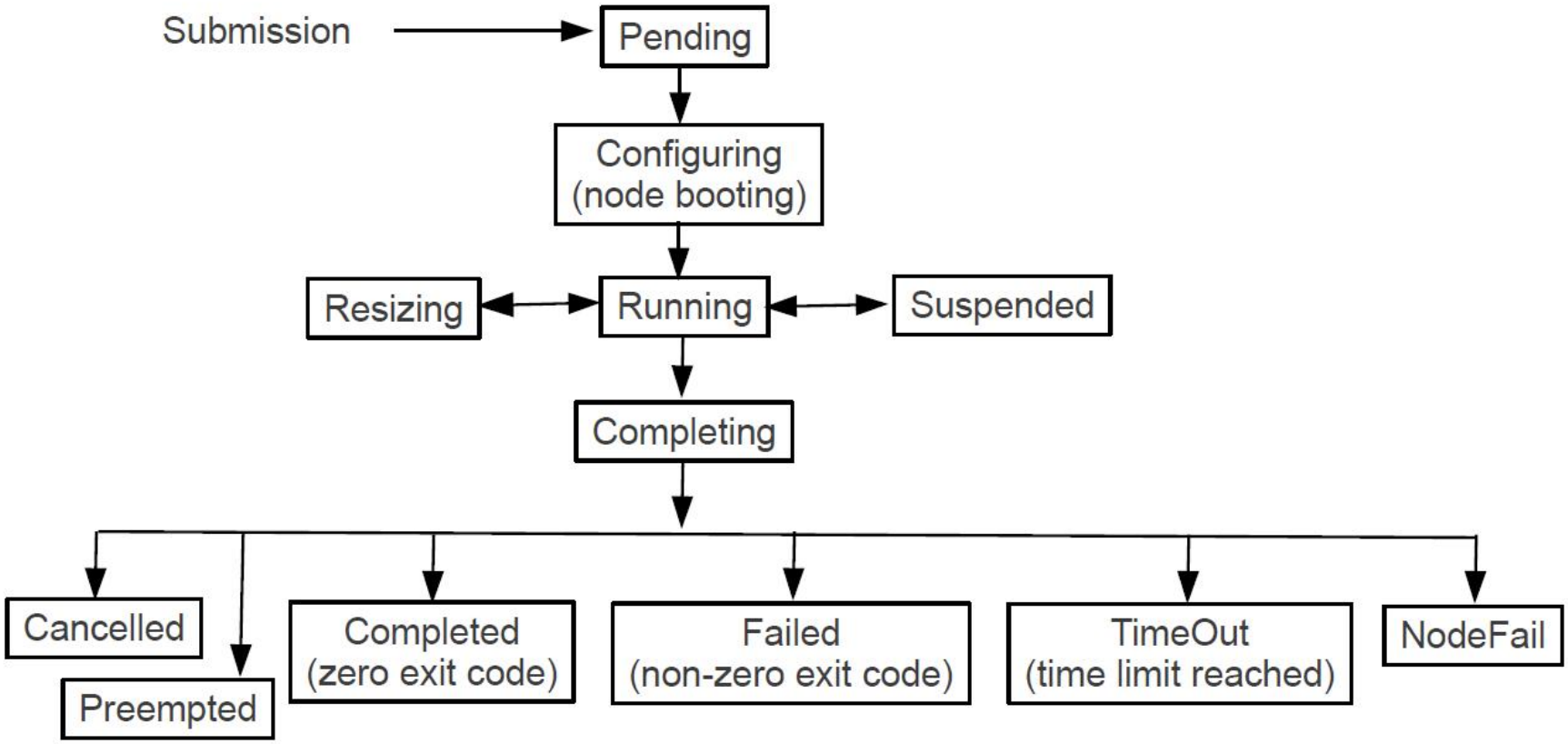
2 如何使用？

3 如何稳定可靠？

集群计算机的核心在软件



集群计算机的核心在软件



SchedMD LLC
http://www.schedmd.com

管理集群计算环境


行业
气象
制造
EDA
教育科研
生命科学
工具套件
集群管理与调度（DonauKit）
集群管理平台
多瑙套件
多瑙Portal
作业调度器
多瑙套件
多瑙调度器
鲲鹏开发套件
DevKit
开发
编译调试
测试
调优&诊断
基础软件（HPCKit）
通信库
Hyper MPI
数学库
鲲鹏KML
编译器
鲲鹏毕昇/GCC编译器
操作系统
openEuler/其他
硬件平台
计算
多样性算力：鲲鹏、xPU
存储
网络
AI Fabric (RoCE)
基础设施
数据中心
液冷

清华开展的集群计算机研制工作


可扩展工作站机群系统研制成功

新华社北京6月13日电

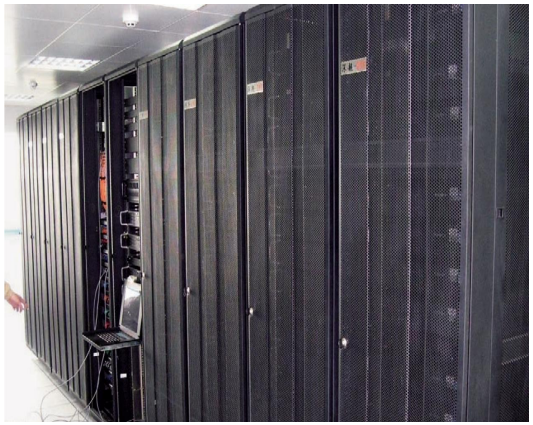
中美在京举行

新华社北京6月13日电

外交

香港事务







1996年
国内率先研制成功

1999年
实现产品化

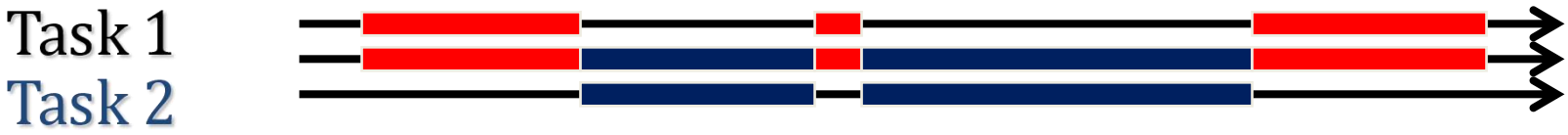
开展了一系列集群计算系统的研制
2003年，深超21C，TOP500第163名
2004年，自强3000，TOP500第126名
2010年，探索100，TOP500第136名

1999年首次并成功用于
北京市气象局国庆50周
年庆祝活动天气保障

建立和运行清华大学高性能计算平台，支持
校内外42个院系单位、259余课题组的大规
模计算需求，效果良好

用好集群计算机基础：并行计算

- 串行处理：多个(子)任务在处理单元上依次执行



- 并行处理：多个(子)任务在多个处理单元上同时执行



并行计算的目标

- 同一个问题可以算的更快
  - 分而治之，每个部分可以更快完成
- 可以利用更多计算资源求解更大的问题
  - 有更多的计算和存储资源，每个计算资源计算相同或相似的子问题，协调完成更大的问题求解
- 提供并发性
  - 例如，搜索引擎

☐ Google query attributes (~2003)
- 150M queries/day (2000/second)
- 100 countries
- 8.0B documents in the index



并行计算机的衡量单位

计算能力的衡量单位 (FLOPS)		
1Mflop/s	1Megaflop/s	10^6flop/s
1Gflop/s	1Gigaflop/s	10^9flop/s
1Tflop/s	1Teraflop/s	10^{12}flop/s
1Pflop/s	1Petaflop/s	10^{15}flop/s
1Eflop/s	1Exaflop/s	10^{18}flop/s
1Zflop/s	1Zettaflop/s	10^{21}flop/s

存储能力的衡量单位		
1MB	1Megabytes	2^{20}Bytes
1GB	1 Gigabytes	2^{30}Bytes
1TB	1 Terabytes	2^{40}Bytes
1PB	1 Petabytes	2^{50}Bytes
1EB	1 Exabytes	2^{60}Bytes
1ZB	1 Zettabytes	2^{70}Bytes

超级计算机(~2.8Eflop/s 双精度, 5.6PB)

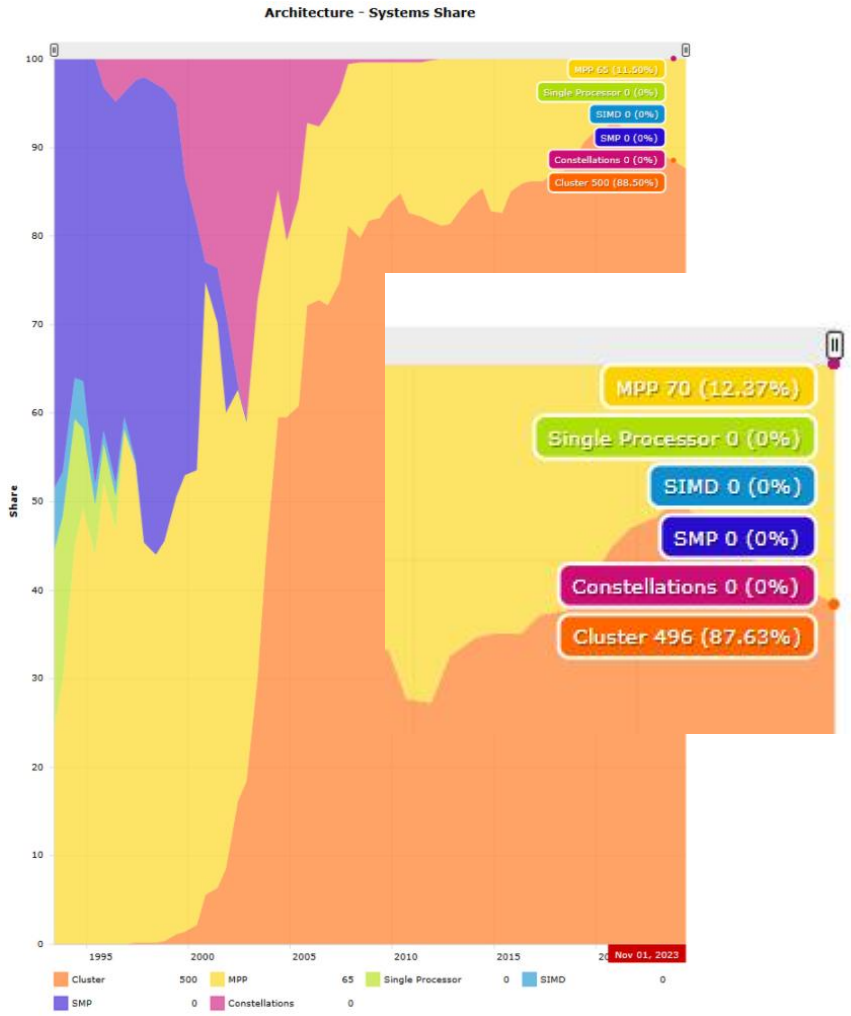
超级计算机TOP500项目

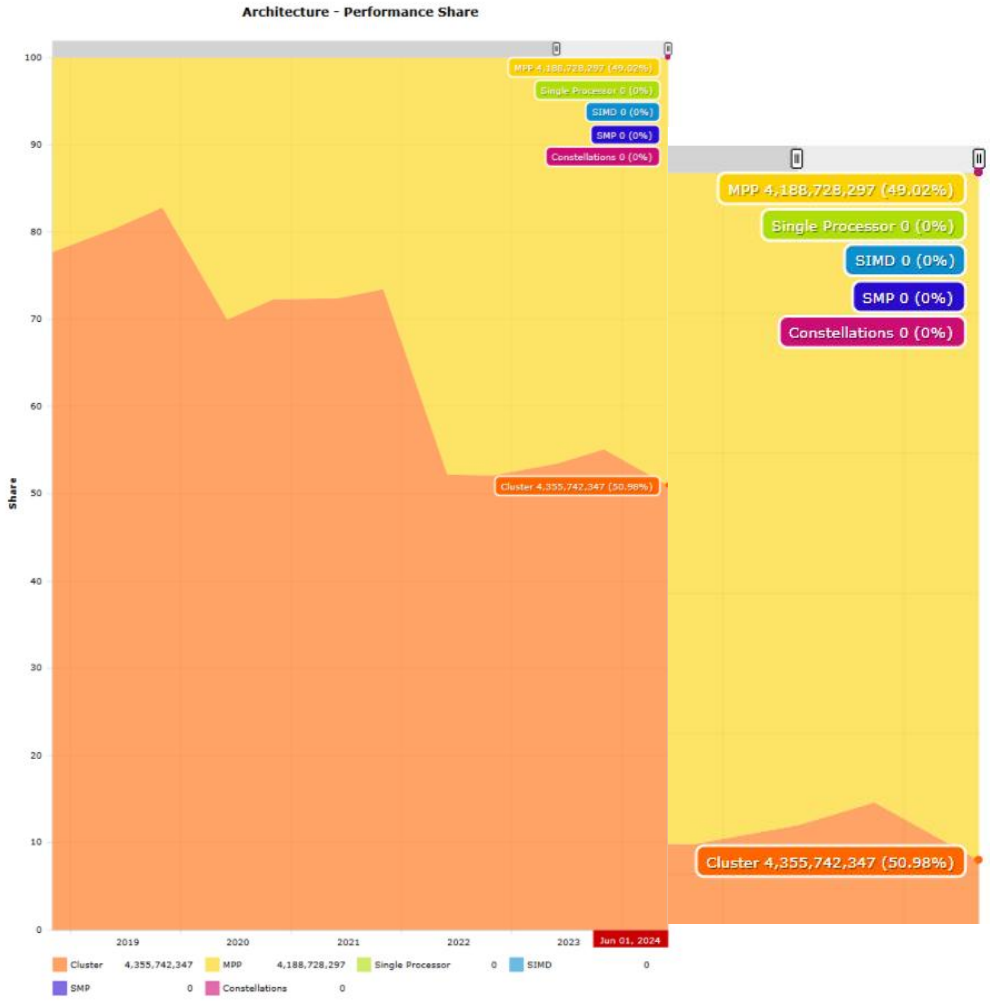
- 为世界上最快的超级计算机排名（前500台）
- 排名依据：Rmax of Linpack
  - 求解大规模稠密线性方程组, 双精度浮点运算，$O(N^3)$ / 求解时间
- 每年发布两次:
  - 6月份在德国法兰克福的ISC国际会议
  - 11月份在美国的SC国际会议
- 排名结果和最新技术进展：www.top500.org

超级计算机TOP500排名（2025.11）

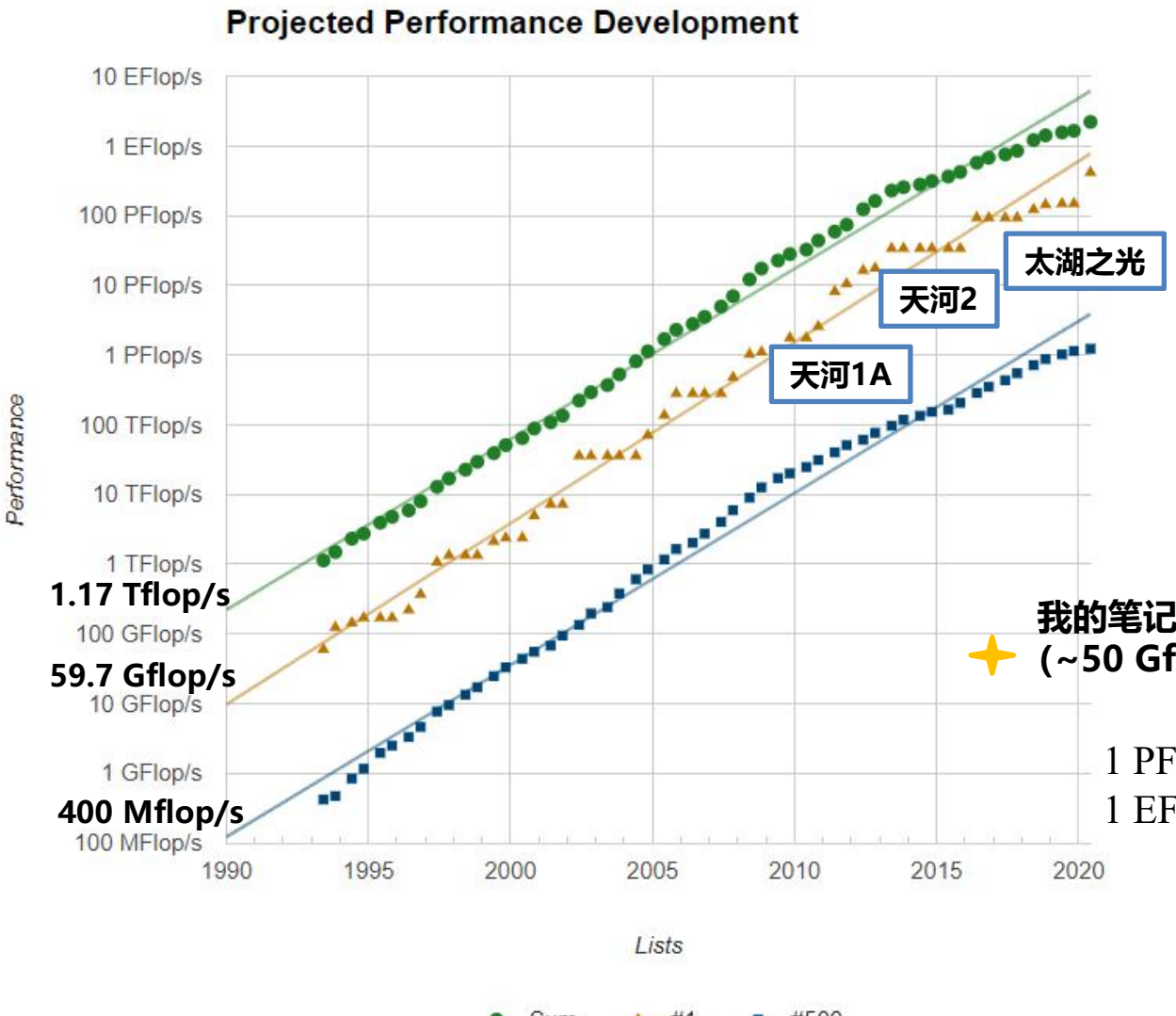
TOP500 2024.12	机器名称	峰值 (PFLOPS)	HPL 效率	架构	加速器类型	功耗 (MW)
1*	El Capitan（美）	**2,821.10**	64.1%	加速器异构	AMD Instinct MI300A	**29.7**
2	Frontier（美）	**2,055.72**	65.8%	加速器异构	AMD Instinct MI250X	**24.6**
3	Aurora（美）	**1,980.01**	51.1%	加速器异构	Intel Data Center GPU Max	**38.7**
	JUPITER Booster（德）	**1226.28**	81.5%	加速器异构	NVIDIA GH200	**15.8**
4*	Eagle（美）	**846.84**	66.3%	加速器异构	NVIDIA H100	-
5	HPC6（意）	**606.97**	78.7%	加速器异构	AMD Instinct MI250X	**8.5**
6	富岳（日）	**537.21**	82.3%	同构	SVE	**29.9**
7	Alps（瑞士）	**574.84**	75.7%	加速器异构	NVIDIA GH200	**7.1**

集群计算机统治TOP500





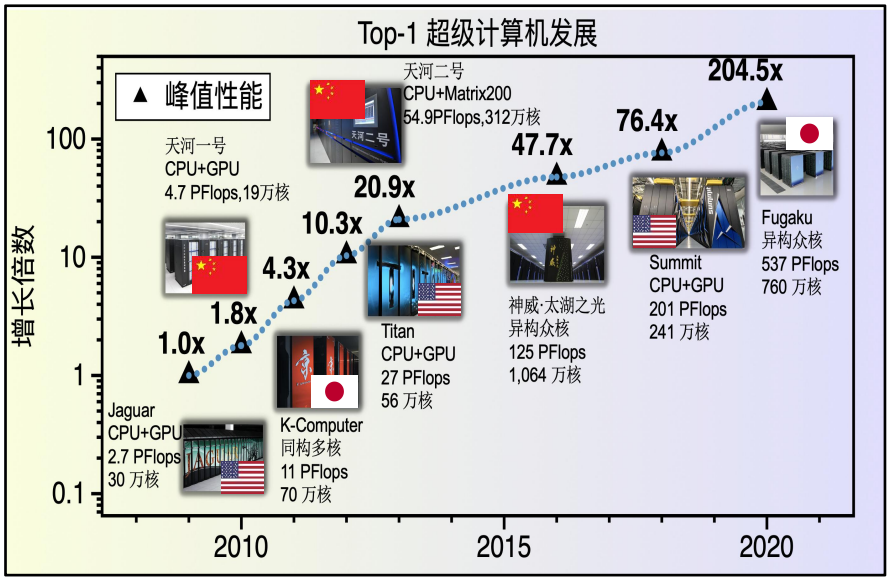
TOP500的性能演变



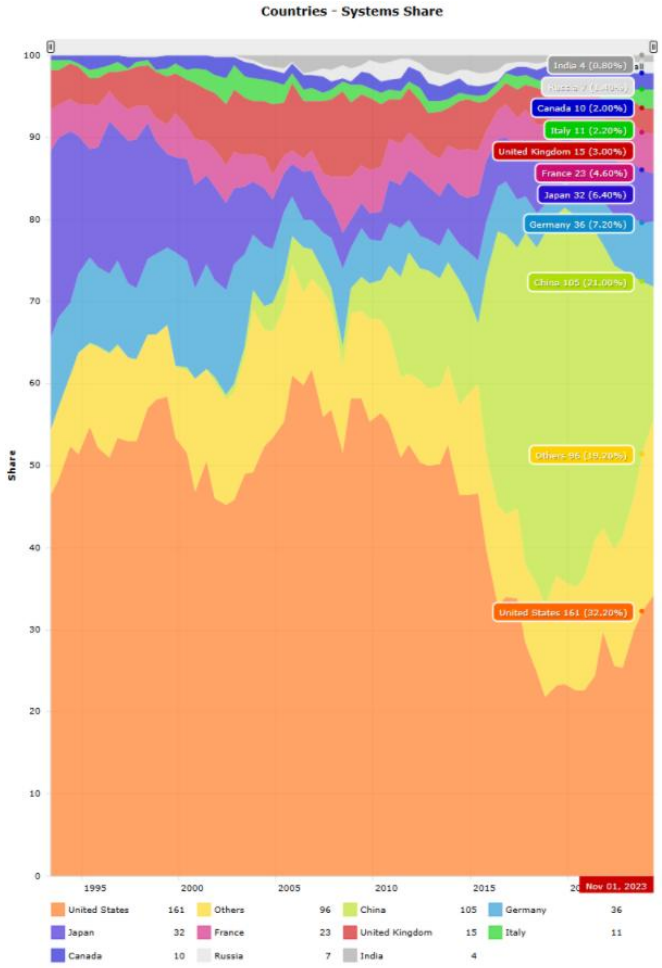
1 PFLOPS=10^{15} DF/s

1 EFLOPS=10^{18} DF/s

高性能计算机中的中国力量



www.top500.org

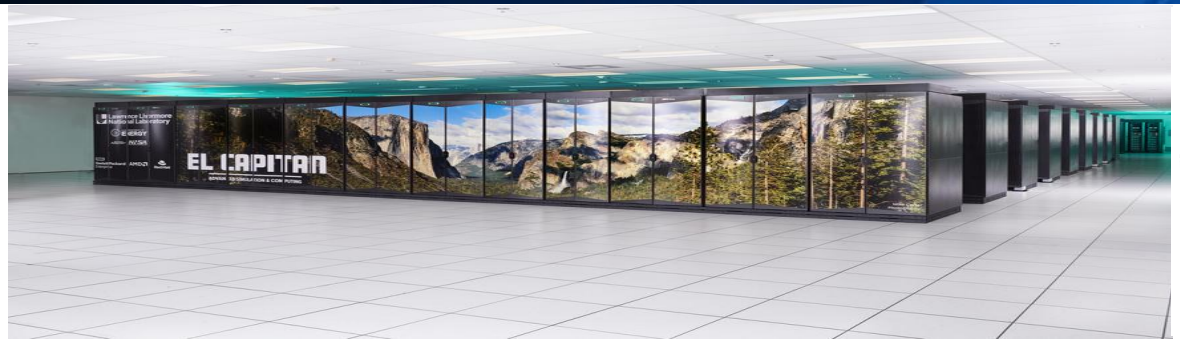


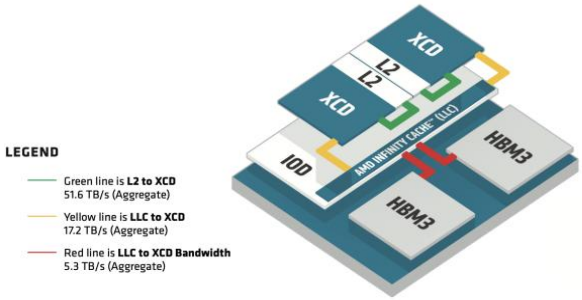
Performance share over Counties/Regions 2024.06

(171)	United States	4,408,382,050	Japan	672,085,510	Finland	391,388,310
(80)	China	356,860,483	Italy	332,618,330	Switzerland	312,137,040
(40)	Germany	274,903,370	France	232,747,500	Spain	221,872,600

我国在顶尖超算系统研制和超算系统部署数量都处于国际前列

El Capitan





	El Capitan
Nodes	11,136
MI300As per node	4
MI300As total	44,544
Node Peak (DP TFLOP/s)	250.8
System Peak (DP PFLOP/s)	2,792.9
System Peak (SP PFLOP/s)	3,688.2
System Peak (HP PFLOP/s)	17,639.4
Node Memory (GiB) (All HBM3)	512
System Memory (TiB)	5,568
Compute cabinets	87
Peak Power (MW)	34.8
Total Rabbit Modules	696
November 2024 Top500 position	1

HPC PEAK THEORETICAL PERFORMANCE (TFLOPS)

FP64 vector	61.3
FP32 vector	122.6
FP64 matrix	122.6
FP32 matrix	122.6

AI PEAK THEORETICAL PERFORMANCE

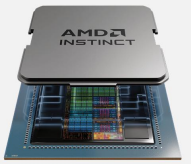
		with sparsity
TF32 matrix (TFLOPs)	490.3	980.6
FP16 (TFLOPs)	980.6	1961.2
BFLOAT16 (TFLOPs)	980.6	1961.2
INT8 (TOPS)	1961.2	3922.3
FP8 (TFLOPS)	1961.2	3922.3

DECODERS AND VIRTUALIZATION

Decoders†	3 groups for HEVC/H.265, AVC/H.264, V1, or AV1
JPEG/MJPEG CODEC	24 cores, 8 cores per group
Virtualization support	SR-IOV, up to 3 partitions

†Video codec acceleration (including at least the HEVC (H.265), H.264, VP9, and AV1 codecs) is subject to and not operable without inclusion/installation of compatible media players. GD-176

SPECIFICATIONS



Form factor	APU SH5 so
Lithography	5nm FinFE
I/O Dies (IODs)	6nm FinFE
AMD 'Zen 4' x86 CPU cores	24
GPU compute units	228
Matrix cores	912
Stream processors	14,592
Peak engine clock	2100 MHz
Memory capacity	128 GB HBM3
Memory bandwidth	5.3 TB/s max. peak theoretical
Memory interface	8192 bits
AMD Infinity Cache™ (last level)	256 MB
Memory clock	5.2 GT/s
Scale-up Infinity Fabric™ Links	4 x16 (128 GB/s)
Scale-out assignable PCIe® Gen 5 or Infinity Fabric Links	4 x16 (128 GB/s)
Scale-out network bandwidth	400 Gbps Ethernet or InfiniBand™
RAS features	Full-chip ECC memory, page retirement, page avoidance
Maximum TDP	550W (air & liquid cooling) 760W (liquid cooling)

TOP500 No.24 神威太湖之光

- ✓ 全世界第一台千万核计算系统，采用异构众核处理器， LINPACK效率74.16%
- ✓ 高密度组装，占地605平米 (FIT楼系统 100Tflop/s，~300平米)
- ✓ 超节点全交换与胖树两级网络，单端口通信带宽56Gbps
- ✓ 水冷系统











运算系统
125.436PFlops
40运算机仓

运算机仓
3.244PFlops
1024处理器

运算超节点
811.008TFlops

运算插件
25.344TFlops

运算节点
6.336TFlops

国产处理器
3.168TFlops

超大规模并行、异构与多精度架构、新型硬件的层次化存储（HBM）成为当前超级计算机的主流

如何计算机的峰值性能

- Peak = Nodes *
  Sockets/nodes *
  Cores/socket *
  Freq. /core *
  FLOP/cycle

指令级并行性
即同时执行多个指令



蓝光：5342 nodes * 2 * 16 * 1.1GHz * 8FLOP = 1.2PF

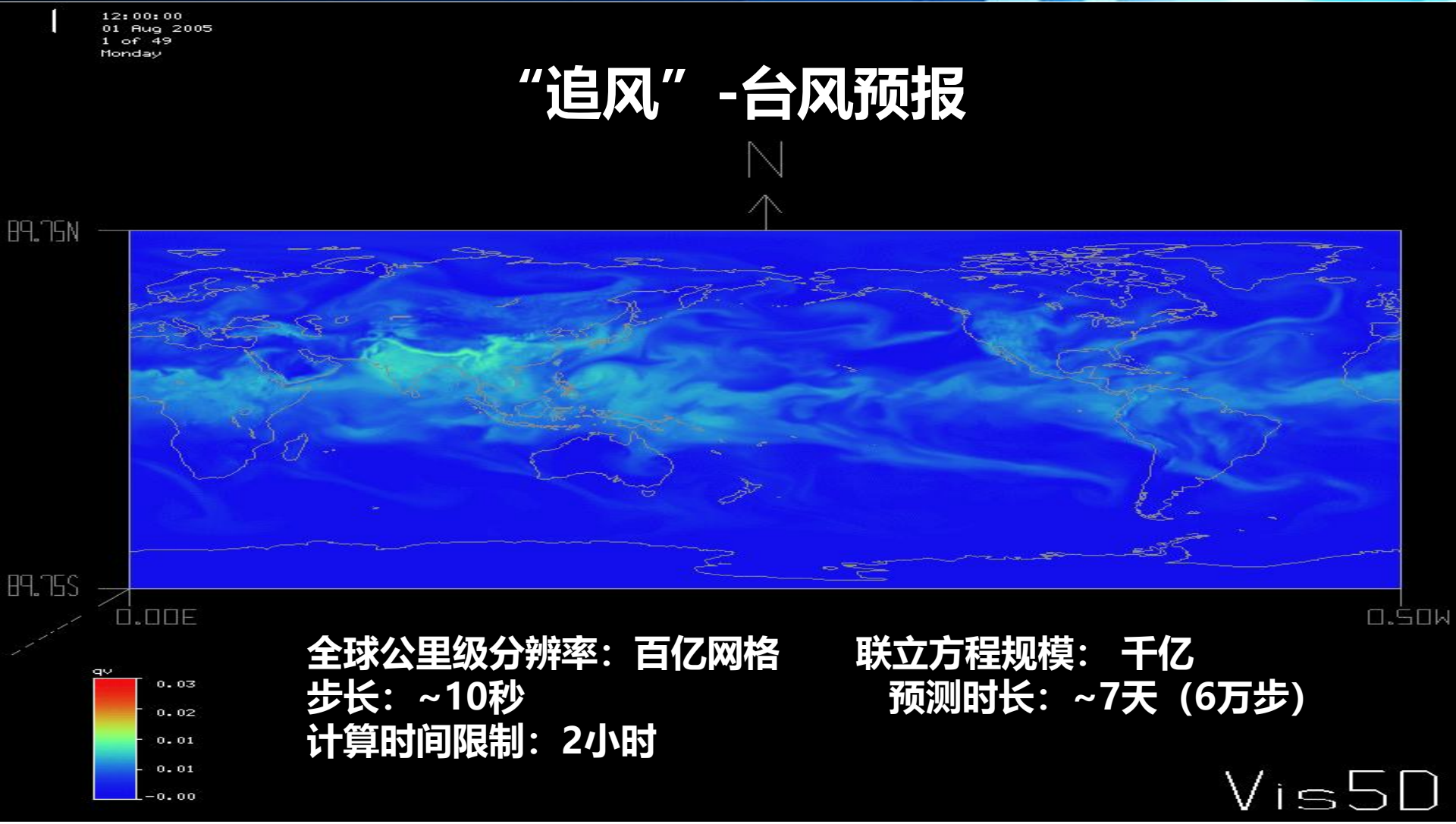
140.8GFLOPS

Intel Xeon X5670 70.32GFLOPS
Fujitsu SPARC VIIIfx 128GFLOPS

课程内容：

- 集群计算机介绍?
- **集群计算机能解决什么问题?**
- 如何在集群计算机上编程序?
- 无所不在的并行处理
- 并行计算的未来挑战

为什么需要集群计算


12:00:00
01 Aug 2005
1 of 49
Monday
“追风”-台风预报
N
89.75N
89.75S
0.00E
0.50W
qv
0.03
0.02
0.01
0.01
-0.00
全球公里级分辨率：百亿网格
步长：~10秒
计算时间限制：2小时
联立方程规模： 千亿
预测时长：~7天（6万步）
Vis5D

为什么需要集群计算

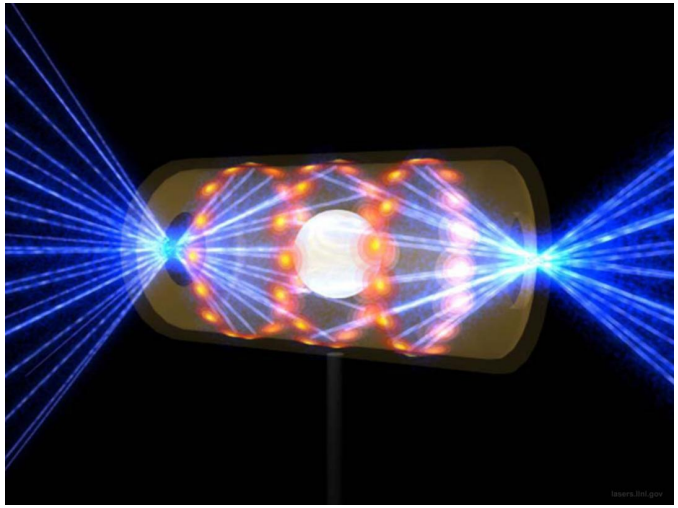


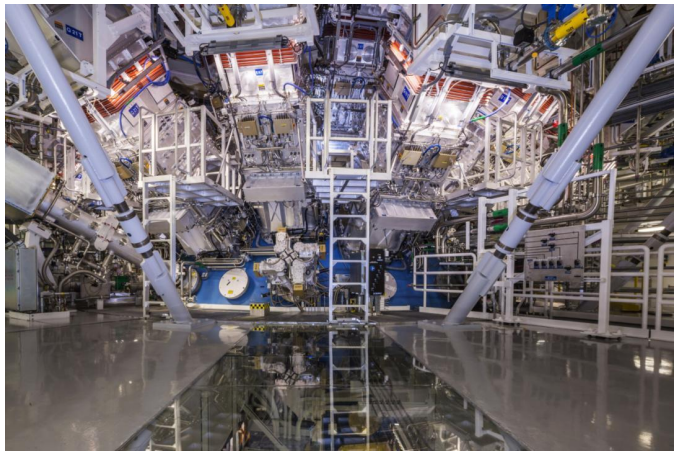
Supercomputing's Critical Role in the Fusion Ignition Breakthrough

By Oliver Peckham

December 21, 2022

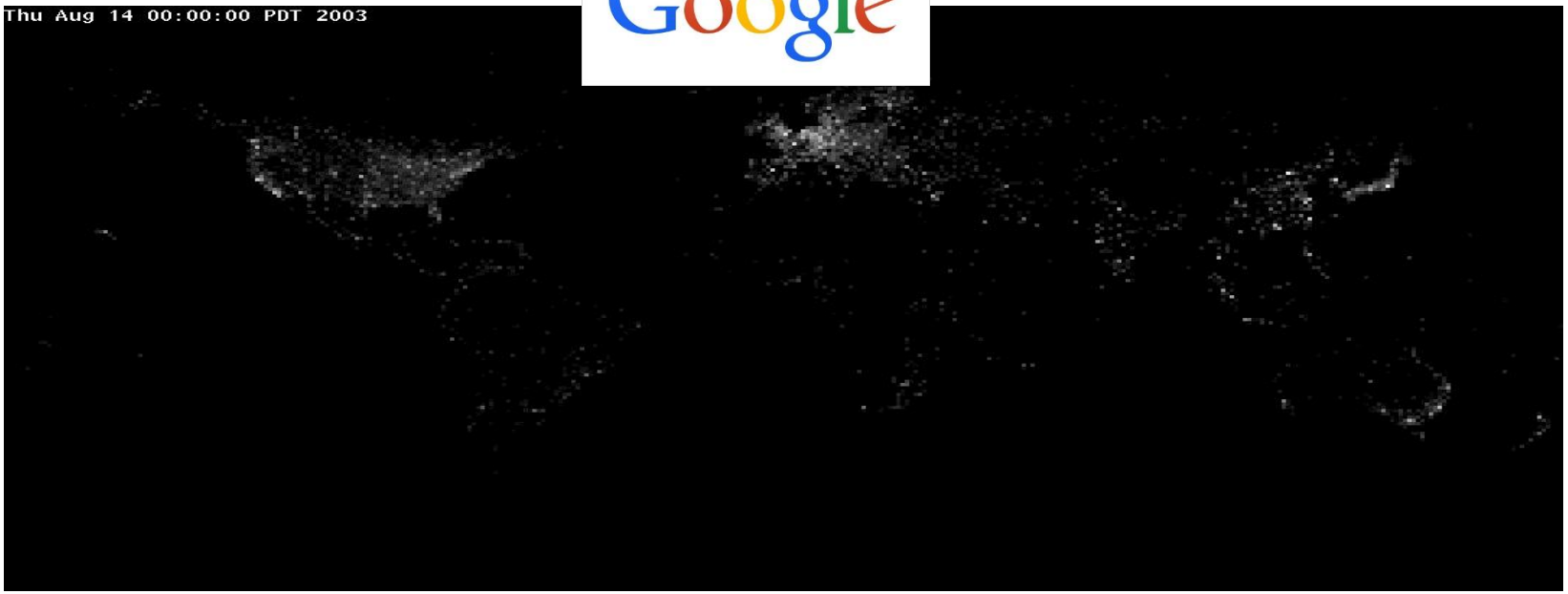
On December 5th, the research team at the National Ignition Facility (NIF) at Lawrence Livermore National Laboratory (LLNL) achieved a historic win in energy science: for the first time ever, more energy was produced by an artificial fusion reaction than was consumed – 3.15 megajoules (MJ) produced versus 2.05 megajoules in laser energy to cause the reaction. High-performance computing was key to this breakthrough (called "ignition"), and *HPCwire* recently had the chance to speak with Brian Spears, deputy lead modeler for inertial confinement fusion (ICF) at the NIF, about HPC's role in making this fusion ignition a reality.





Inertial confinement fusion, 惯性约束聚变

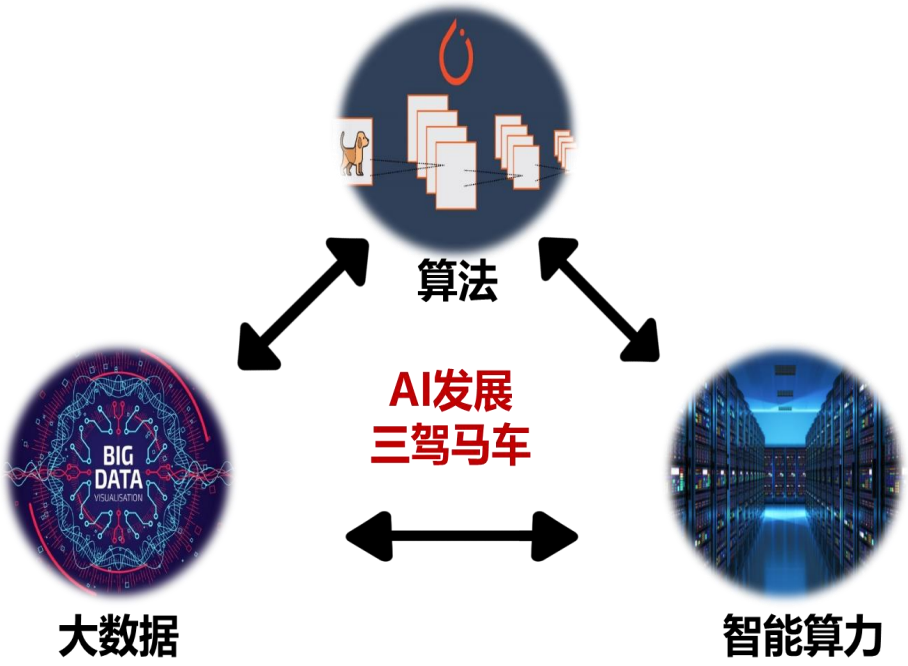
为什么需要集群计算



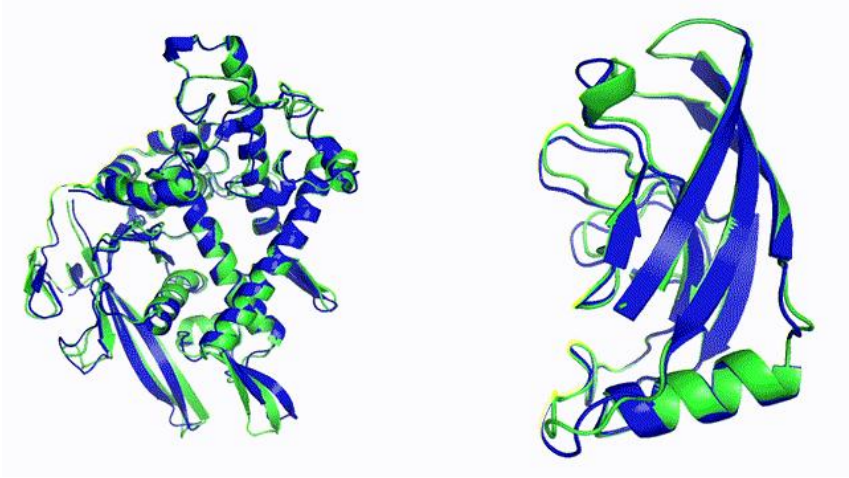
- Google query attributes (~2003)
  - 150M queries/day (2000/second)
  - 100 countries
  - 8.0B documents in the index
- Google infrastructure (~2003)
  - >200,000 Linux servers
  - Storage capacity >5 petabytes
  - Traffic growth 20-30%/month

为什么需要集群计算

强大的算力是AI领域创新的动力



AlphaFold2解决了结构生物学50年难题



[Senior, et. al. Nature. 2020]

为什么需要集群计算

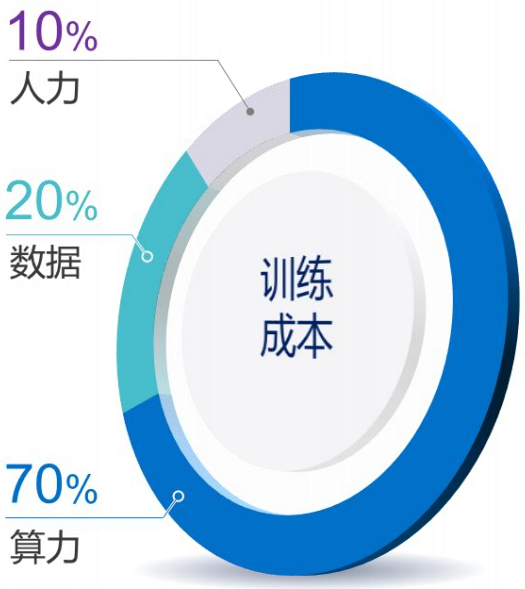
强大的算力和高效的算力利用是AI领域创新的动力

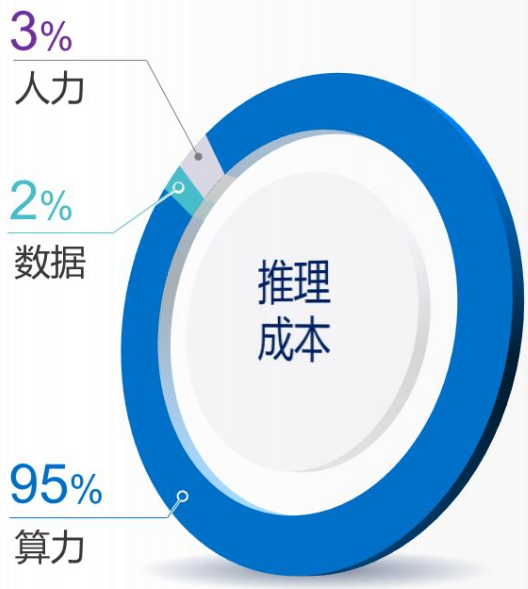
200 万美元/每月
模型研发成本
800 块 A100

2 亿美元+
GPT4 训练开销
一万块 A100

70 万美元/每天
ChatGPT 推理开销

70% 以上算力成本占大模型企业开销





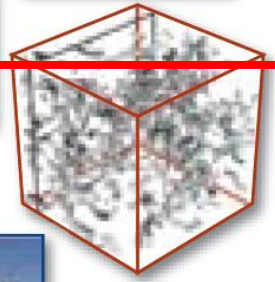
为什么需要集群计算

Science Paradigms

- Thousand years ago:
  science was **empirical**
  describing natural phenomena
- Last few hundred years:
  theoretical branch
  using models, generalizations
- Last few decades:
  a **computational** branch
  simulating complex phenomena
- Today: **data exploration** (eScience)
  unify theory, experiment, and simulation
  - Data captured by instruments or generated by simulator
  - Processed by software
  - Information/knowledge stored in computer
  - Scientist analyzes database/files using data management and statistics



$$\left(\frac{\dot{a}}{a}\right)^2 = \frac{4\pi G\rho}{3} - K\frac{c^2}{a^2}$$







实验

理论

仿真

数据分析

并行/集群计算

From Jim Gray, 2007

为什么需要集群计算

并行计算可以帮助我们快速求解富有挑战性的计算和数据分析问题：

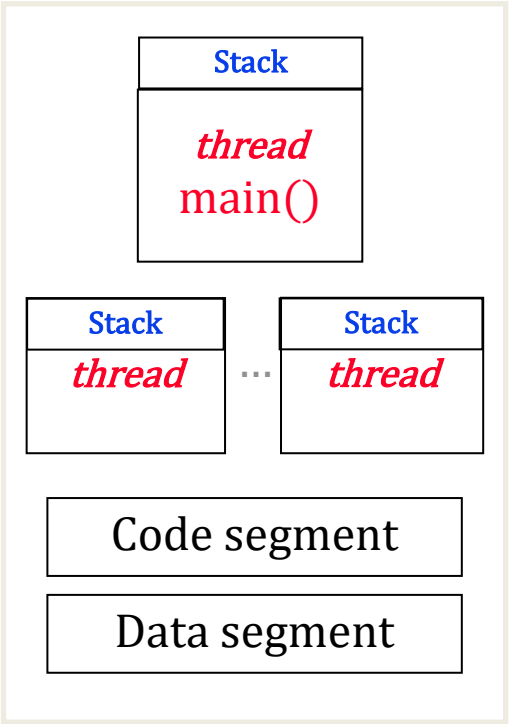
- Science
  - **Global climate modeling**
  - Astrophysical modeling
  - Biology: **genomics**; protein folding; drug design
  - Computational Chemistry
  - Computational Material Sciences and **Nanosciences**
- Business
  - Financial and economic modeling
  - Transaction processing, web services and search engines
- Engineering
  - **Crash simulation**
  - Semiconductor design
  - Earthquake and structural modeling
  - Computation fluid dynamics (airplane design)
  - Combustion (engine design)
  - **Oil field applications**
- Defense
  - **Nuclear weapons** -- test by simulations
  - Cryptography

神机妙算：超级计算机都在算什么?
链接：https://cloud.tsinghua.edu.cn/f/14c73f4061ff42f5bbda/

课程内容：

- 集群计算机介绍?
- 集群计算机能解决什么问题?
- **如何在集群计算机上编程序?**
- 无所不在的并行处理
- 并行计算的未来挑战

并行程序运行基本单元：进程与线程

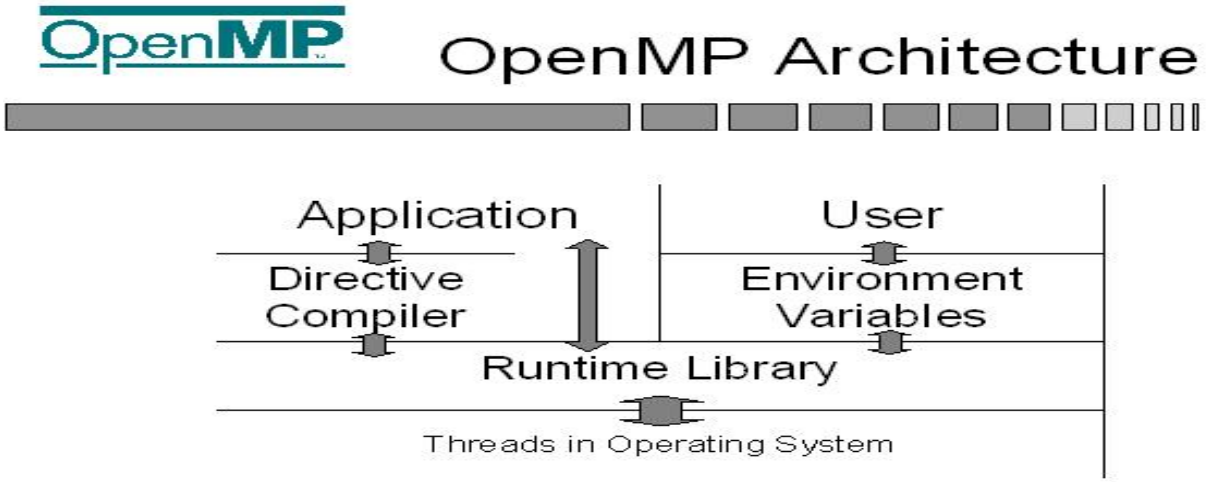


- Modern operating systems load programs as processes
  - Resource holder
  - Execution
- A process starts executing at its entry point as a thread
- Threads can create other threads within the process
  - Each thread gets its own stack
- All threads within a process share code & data segments

可以采用进程和线程实现并行程序

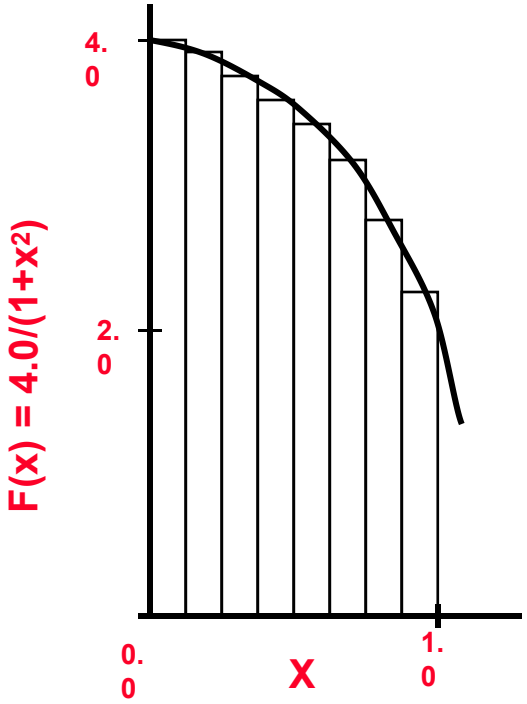
多线程并行编程环境之一OpenMP

- OpenMP是一种面向共享存储的并行程序设计标准
  - 该标准不断演变，当前版本为v6.0（2024.11）
  - 大多针对单节点系统
  - http://www.openmp.org
- 一组编译制导语句和可调用的运行(run-time)库函数, 扩充到基本语言中用来表达程序中的并行性



1. Overview 9

小实验：积分求π



$$\int_0^1 \frac{4.0}{(1+x^2)}\,dx = \pi$$

```
static long num_steps = 1000000000;
double step;
void main ()
{int i;
 double x, pi, sum = 0.0;
    step = 1.0/(double) num_steps;
    for (i=0; i< num_steps; i++){
       x = (i+0.5)*step;
       sum = sum + 4.0/(1.0+x*x);
    }
    pi = step * sum;
}
```

在计算机系的课程集群上串行程序平均运行2123毫秒

积分求π的一个OpenMP实现

```
static long num_steps = 1000000000;
double step;
void main ()
{
 int i;
 double x, pi, sum = 0.0;

    step = 1.0/(double) num_steps;

    #pragma omp parallel for private(x) reduction(+ : sum)
    for (i=0; i< num_steps; i++){
      x = (i+0.5)*step;
      sum = sum + 4.0/(1.0+x*x);
    }

    pi = step * sum;
}
```

线程数	OpenMP (ms)
1	2124
2	1211
4	533
6	379
12	203
24	110

多进程并行编程环境之一MPI

消息通信并行编程标准–Message Passing Interface

- 目前应用最为广泛的消息传递程序设计标准，在集群一个节点和多个节点都可以使用
- 语言绑定：MPI的实现是一个库而不是一种程序语言，因此对MPI的使用必须和特定的程序语言结合起来进行

功能	含义
mpi_init	启动MPI
mpi_finalize	结束MPI计算
mpi_comm_size	确定进程数
mpi_comm_rank	返回进程ID
mpi_send	发送一条消息
mpi_recv	接收一条消息

积分求π的一个MPI实现

```
#include "stdio.h"
#include "mpi.h"
int main(argc, argv)
int argc;
char* argv[];
{
int i, group_size, my_rank;
long N;
double x, sum=0.0, step, tmp;
MPI_Status status;
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &group_size);
MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
if (my_rank==0) {
      printf("Enter N:"); scanf("%d",&N);
      for (i=1;i<group_size;i++)
            MPI_Send(&N, 1, MPI_LONG, i, i, MPI_COMM_WORLD);
      step = 1.0/(double) N;
      for (i=my_rank;i<N;i=i+group_size)
          {x = (i+0.5)*step; sum=sum+4.0/(1.0+x*x); }
      for (i=1;i<group_size;i++) {
            MPI_Recv(&tmp,1,MPI_DOUBLE,i,i,MPI_COMM_WORLD,&status);
            sum=sum+tmp;
      }
      printf("\n The result = %d\n", sum*step);
}
else {
      MPI_Recv(&N,1,MPI_LONG,0,my_rank,MPI_COMM_WORLD,&status);
      step = 1.0/(double) N;
      for (i=my_rank;i<N;i=i+group_size)
          {x = (i+0.5)*step; sum=sum+4.0/(1.0+x*x); }
      MPI_Send(&sum,1,MPI_DOUBLE,0,my_rank,MPI_COMM_WORLD);
}
MPI_Finalize();
return 0; }
```

初始化MPI环境

得到缺省的进程组大小

得到每个进程在组中的编号

发送消息

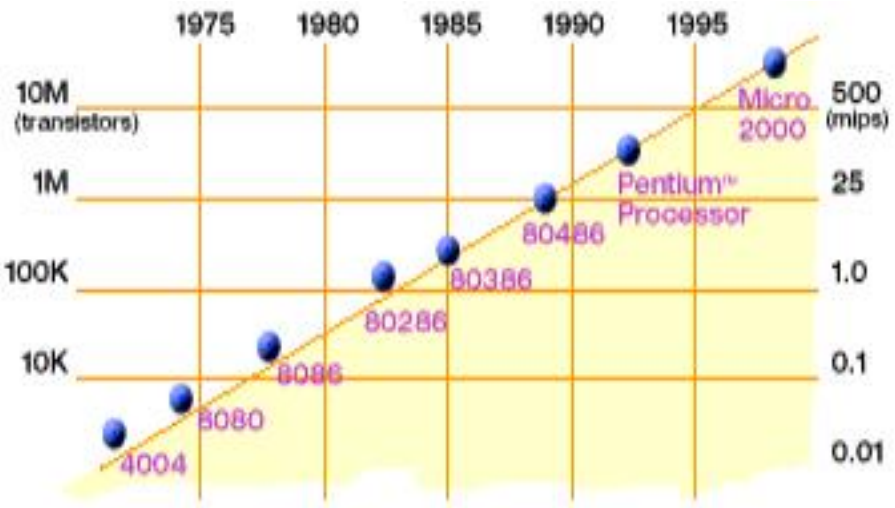
接收消息

终止MPI环境

课程内容：

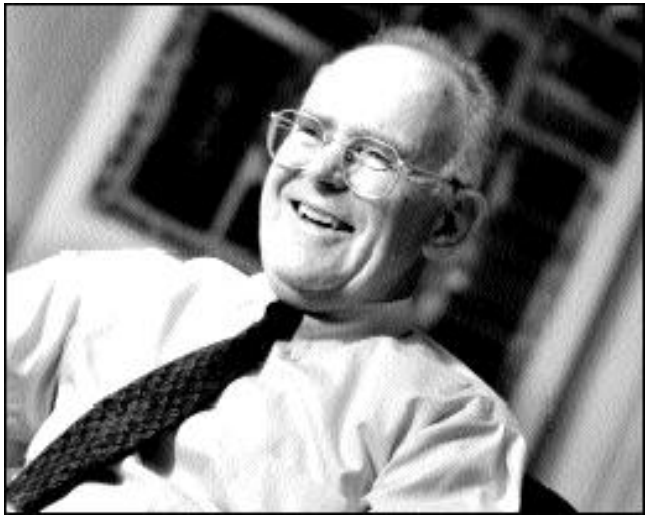
- 集群计算机介绍?
- 集群计算机能解决什么问题?
- 如何在集群计算机上编程序?
- **无所不在的并行处理**
- 并行计算的未来挑战

摩尔定律



2X transistors/Chip Every 1.5 years
Called "Moore's Law"

Microprocessors have become smaller, denser, and more powerful.

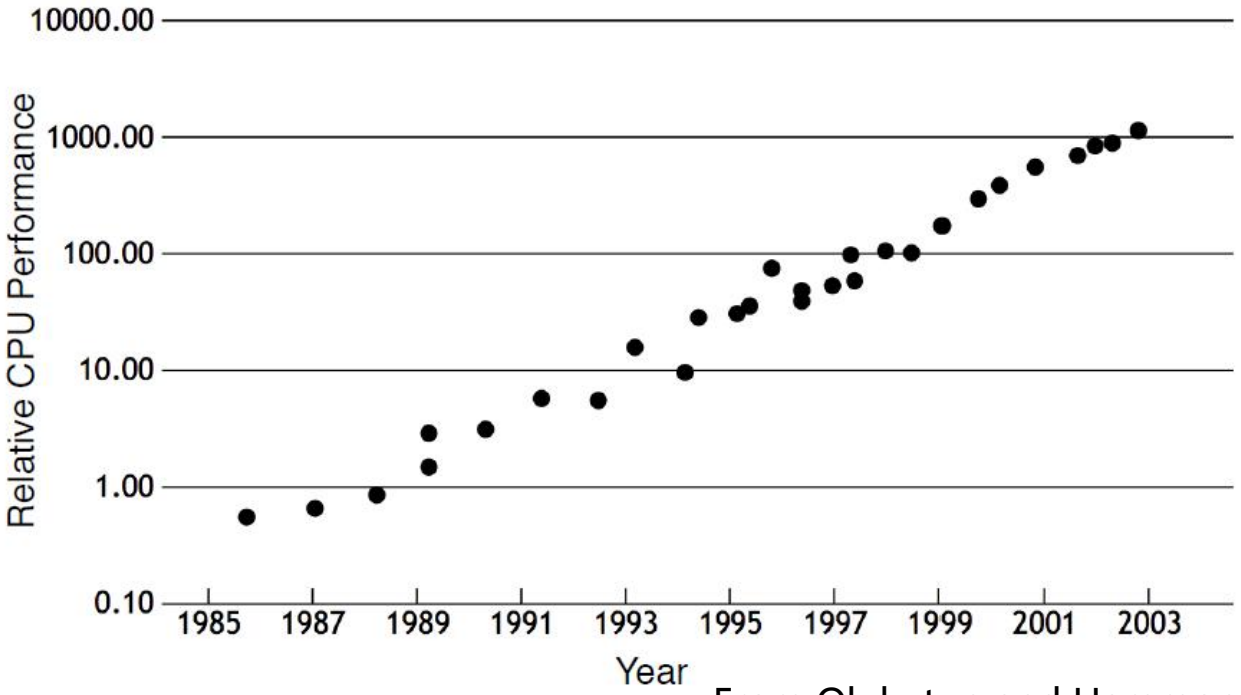


Gordon Moore (co-founder of Intel) predicted in 1965 that the transistor density of semiconductor chips would double roughly every 18 months.

From Jack Dongarra

历史上的摩尔定律红利

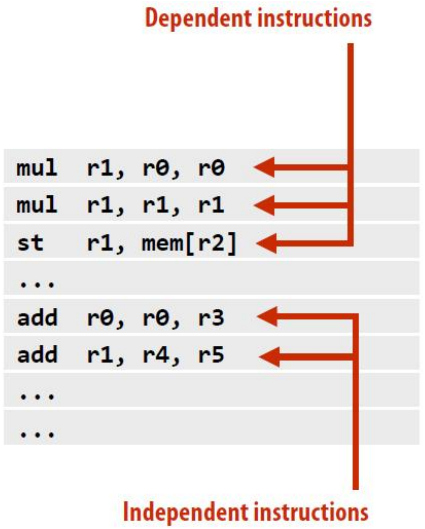
- 单处理器的性能随摩尔定律而提升
  - **处理器频率提升、开发指令级并行性**
- 程序员可以坐享其成，无需并行你的应用☺



From Olukutun and Hammond, 2005

指令级并行性

- 单处理器中同时执行多个指令，加速应用程序执行，充分利用处理器计算资源，对程序员透明
  - 基本方法：在程序指令序列中寻找非依赖的指令并行执行，确保程序执行正确性
  - 指令级并行的种类
    - 超标量执行（Superscalar）
    - 流水线执行（Pipeline）
    - 超流水执行（Superpipeline）
    - **向量化执行（Vectorization）**

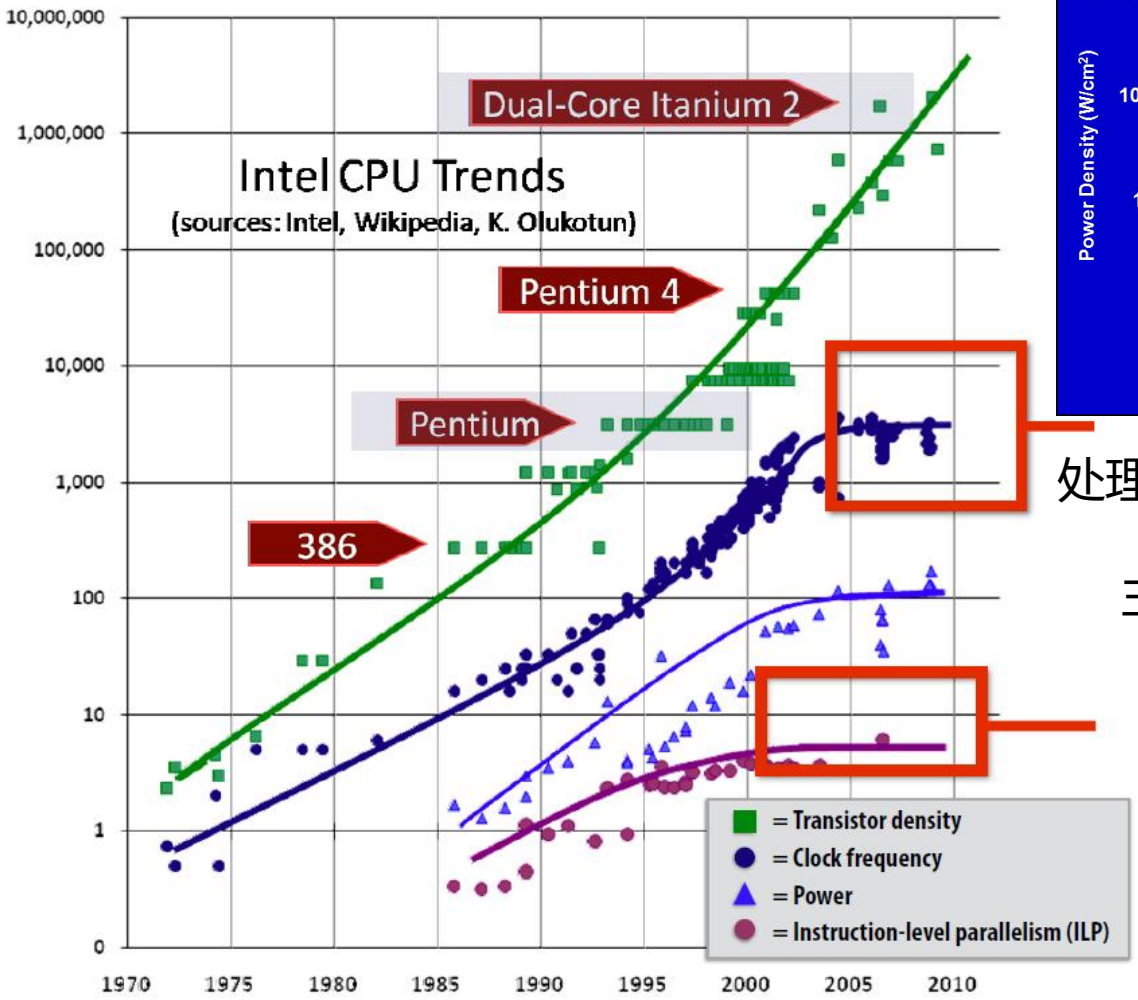


Basic five-stage pipeline

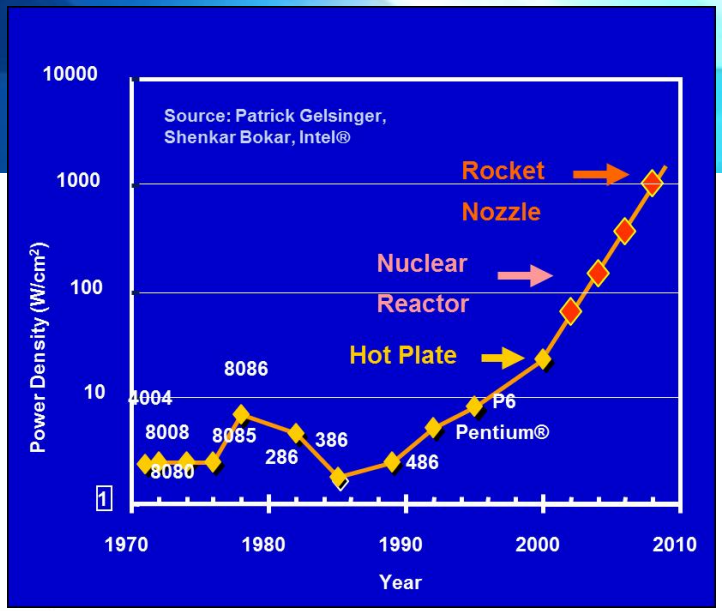
Instr. No. \ Clock cycle	1	2	3	4	5	6	7
1	IF	ID	EX	MEM	WB		
2		IF	ID	EX	MEM	WB	
3			IF	ID	EX	MEM	WB
4				IF	ID	EX	MEM
5					IF	ID	EX

(IF = Instruction Fetch, ID = Instruction Decode, EX = Execute, MEM = Memory access, WB = Register write back).

红利的终结



From Herb Sutter, 2005



处理器功耗=$V^2 * f * C + V * I_{leakage}$

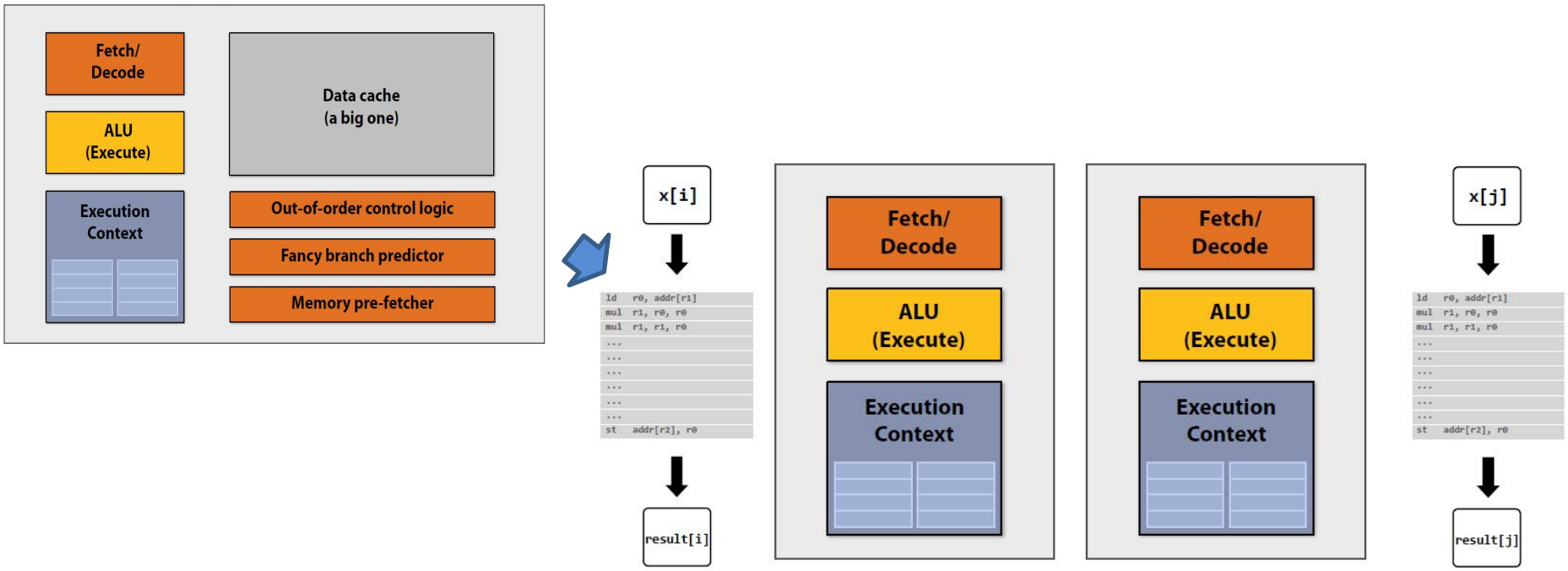
提升频率f同时提升电压V

主频受功耗限制难以继续提升

指令级并行性无法得到更好的性能提升

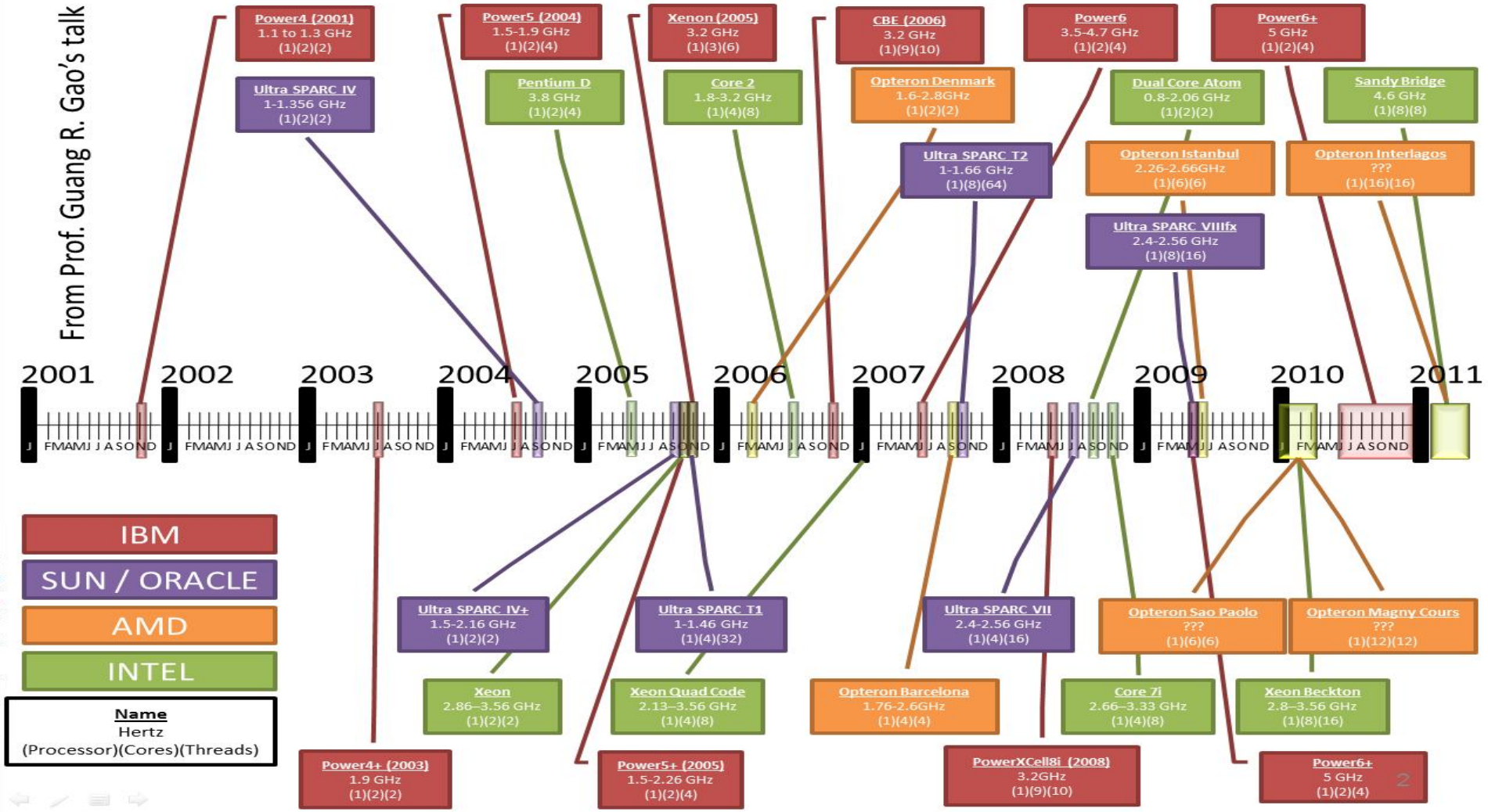
核趋向简单、
核数增加成为趋势
功耗不再指数增加

从单核走向多核处理器



- 采用更为简单的核设计，单核性能较复杂核有所降低，例如性能降低25%
- 多核处理器同时支持多条指令流执行，理论提速核数倍
- **潜在加速效果：2 * 0.75 = 1.5倍**

并行处理无所不在！


From Prof. Guang R. Gao's talk
Power4 (2001)
1.1 to 1.3 GHz
(1)(2)(2)
Ultra SPARC IV
1-1.356 GHz
(1)(2)(2)
Power5 (2004)
1.5-1.9 GHz
(1)(2)(4)
Pentium D
3.8 GHz
(1)(2)(4)
Xenon (2005)
3.2 GHz
(1)(3)(6)
Core 2
1.8-3.2 GHz
(1)(4)(8)
CBE (2006)
3.2 GHz
(1)(9)(10)
Opteron Denmark
1.6-2.8GHz
(1)(2)(2)
Power6
3.5-4.7 GHz
(1)(2)(4)
Power6+
5 GHz
(1)(2)(4)
Dual Core Atom
0.8-2.06 GHz
(1)(2)(2)
Sandy Bridge
4.6 GHz
(1)(8)(8)
Ultra SPARC T2
1-1.66 GHz
(1)(8)(64)
Opteron Istanbul
2.26-2.66GHz
(1)(6)(6)
Opteron Interlagos
???
(1)(16)(16)
Ultra SPARC VIIIfx
2.4-2.56 GHz
(1)(8)(16)
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
J FMAMJ J A S O N D
IBM
SUN / ORACLE
AMD
INTEL
Name
Hertz
(Processor)(Cores)(Threads)
Ultra SPARC IV+
1.5-2.16 GHz
(1)(2)(2)
Xeon
2.86–3.56 GHz
(1)(2)(2)
Power4+ (2003)
1.9 GHz
(1)(2)(2)
Ultra SPARC T1
1-1.46 GHz
(1)(4)(32)
Xeon Quad Code
2.13–3.56 GHz
(1)(4)(8)
Power5+ (2005)
1.5-2.26 GHz
(1)(2)(4)
Opteron Barcelona
1.76-2.6GHz
(1)(4)(4)
Ultra SPARC VII
2.4-2.56 GHz
(1)(4)(16)
PowerXCell8i (2008)
3.2GHz
(1)(9)(10)
Opteron Sao Paolo
???
(1)(6)(6)
Core 7i
2.66–3.33 GHz
(1)(4)(8)
Opteron Magny Cours
???
(1)(12)(12)
Xeon Beckton
2.8–3.56 GHz
(1)(8)(16)
Power6+
5 GHz
(1)(2)(4)

并行处理无所不在！

- 并行系统已经逐渐成为主流
  - 多核、众核已经成为趋势

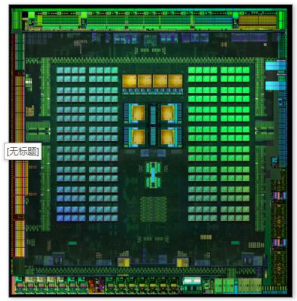


图形处理单元GPU

移动处理器

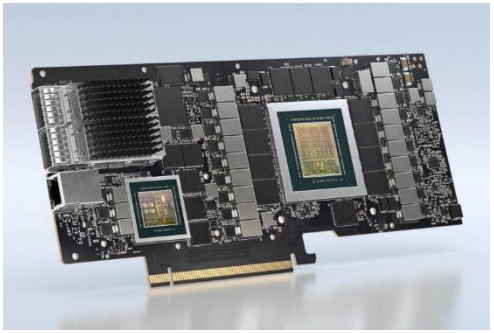


Apple A9: (in iPhone 6s)
Dual-core CPU + GPU + image processor and more on one chip

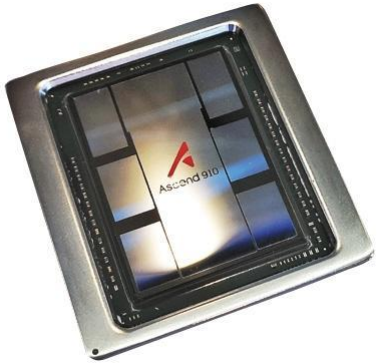


NVIDIA Tegra X1:
4 ARM A57 CPU cores +
4 ARM A53 CPU cores +
NVIDIA GPU + image processor...

数据处理单元DPU

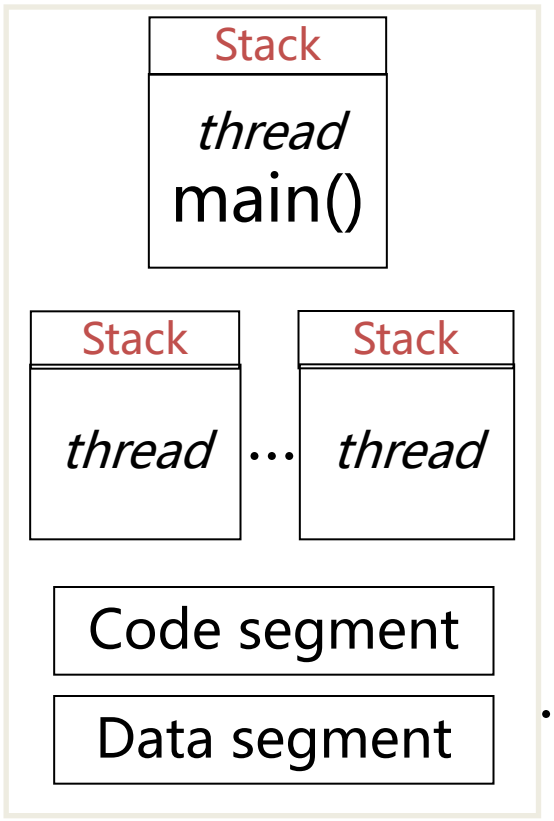


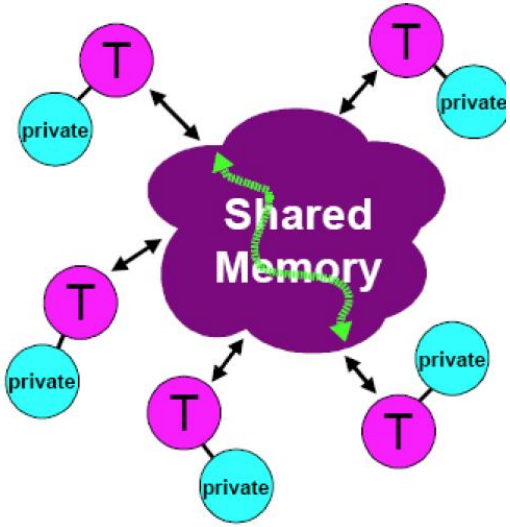
神经网络处理单元NPU



并行计算已经成为通用计算技术

软件挑战：并行程序设计



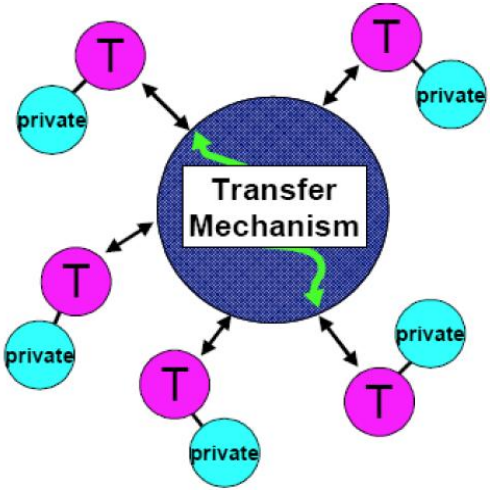


共享存储并行编程模型

- 线程间拥有共享数据
- 数据交互即读写变量
- **显式同步保证计算顺序**
- Pthreads、OpenMP Windows Threads

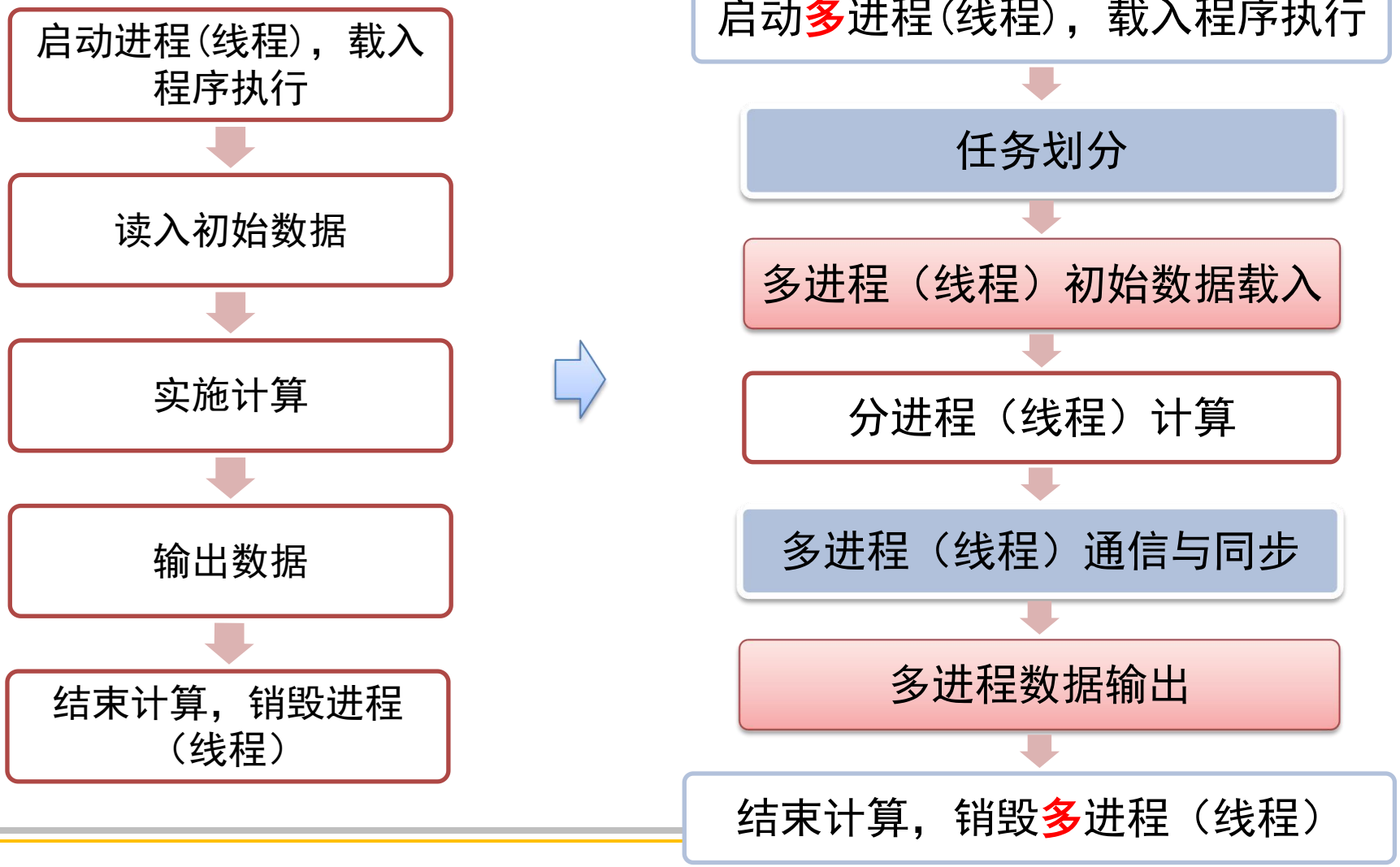
消息传递并行编程模型

- 数据全部私有
- **显式通信达到交互**
- 通信同时达成同步
- Socket，MPI



采用进程和线程由程序员实现并行程序

并行程序设计

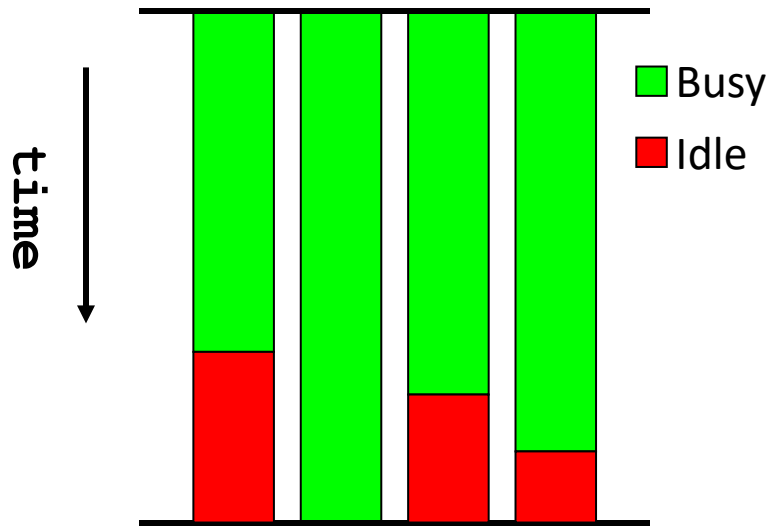

启动进程(线程)，载入程序执行
读入初始数据
实施计算
输出数据
结束计算，销毁进程（线程）
启动多进程(线程)，载入程序执行
任务划分
多进程（线程）初始数据载入
分进程（线程）计算
多进程（线程）通信与同步
多进程数据输出
结束计算，销毁多进程（线程）

并行程序的潜在性能问题

过高的并行成本

划分引入的负载不均衡

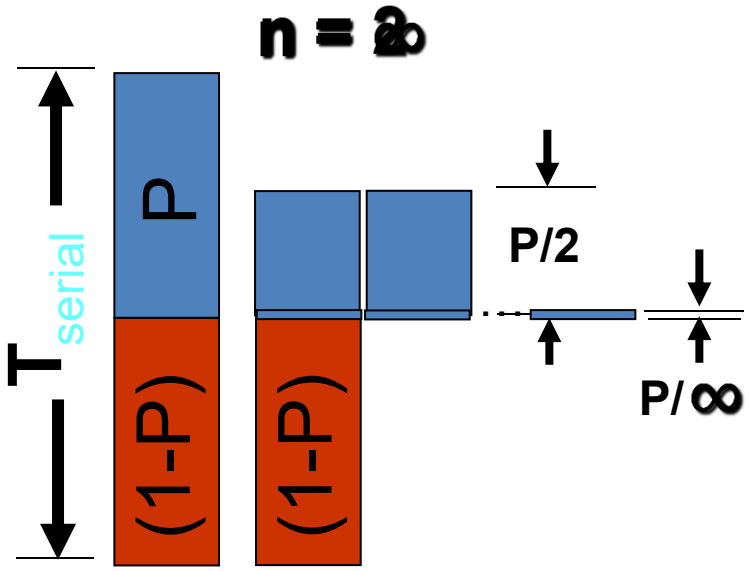
L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	25 ns
Main memory reference	100 ns
Compress 1K bytes with Zippy	3,000 ns
Send 2K bytes over 1 Gbps network	20,000 ns
Read 1 MB sequentially from memory	250,000 ns
Round trip within same datacenter	500,000 ns
Disk seek	10,000,000 ns
Read 1 MB sequentially from disk	20,000,000 ns
Send packet CA->Netherlands->CA	150,000,000 ns



A list of numbers every programmer should know (Jeff Dean, google)

并行程序的潜在性能问题

缺乏足够的并行部分



0.5 + 0.25

$$T_{parallel} = \{(1-P) + P/n\} T_{serial}$$

n = number of processors

Speedup = $T_{serial} / T_{parallel}$ 1.0/0.75 = 1.33

Amdahl’ s Law

免费午餐不再！

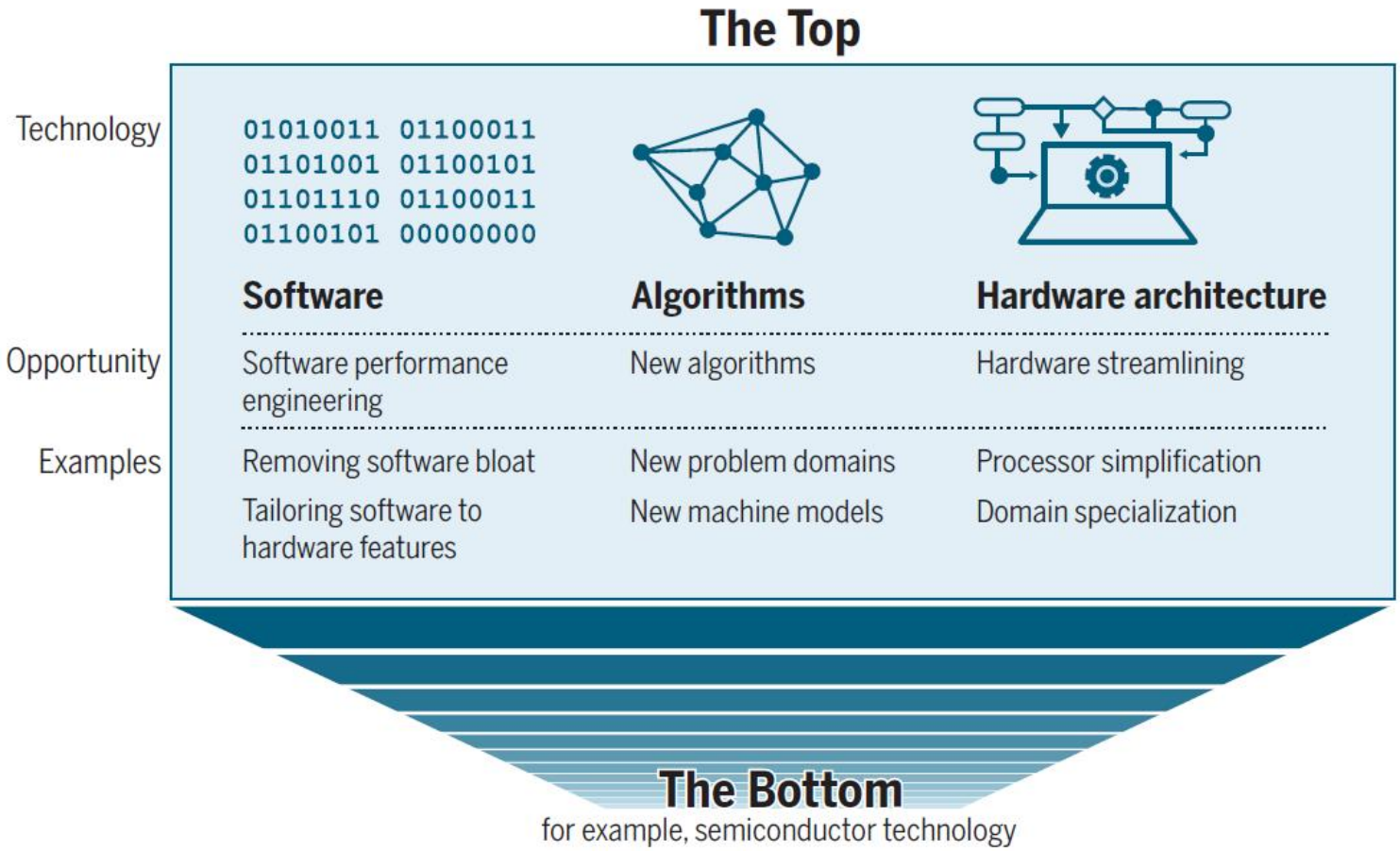
- 性能自动提升已经成为历史

- 并行无所不在，成为通用计算技术
  - 多核和众核已经成为趋势，超级计算机、服务器、桌面系统和手机等均为并行系统，且核数不断增加
  - 需要**编写高效的并行程序**来进一步提升性能

- 并行程序设计和性能工程应成为每位程序员的必修课
  - Thinking of Parallelism：好的串行算法不等于好的并行算法
  - 深入理解并行程序设计原则：划分、通信和同步等
  - 深入理解计算机体系结构：从单核到多核、众核到并行系统（UMA、NUMA、NVLink、CXL、InfiniBand、UB/灵衢等）

需要掌握新知识和技能，充分利用计算资源，让你的程序跑的多快好省！

后摩尔定律的性能创新



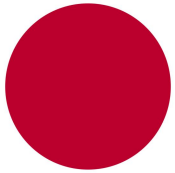
There's plenty of room at the Top: What will drive computer performance after Moore's law? Science, 2020.6

课程内容：

- 集群计算机介绍?
- 集群计算机能解决什么问题?
- 如何在集群计算机上编程序?
- 无所不在的并行处理
- **并行计算的未来挑战**

Zetta级（1000E）系统竞争





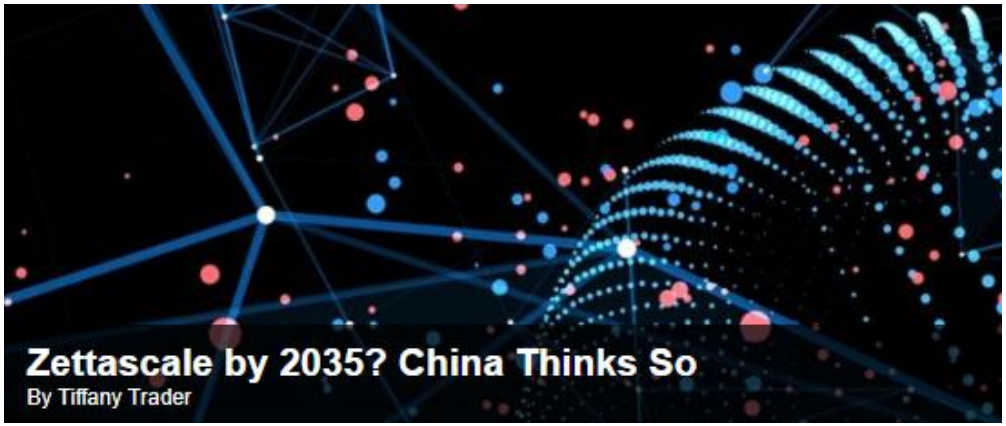
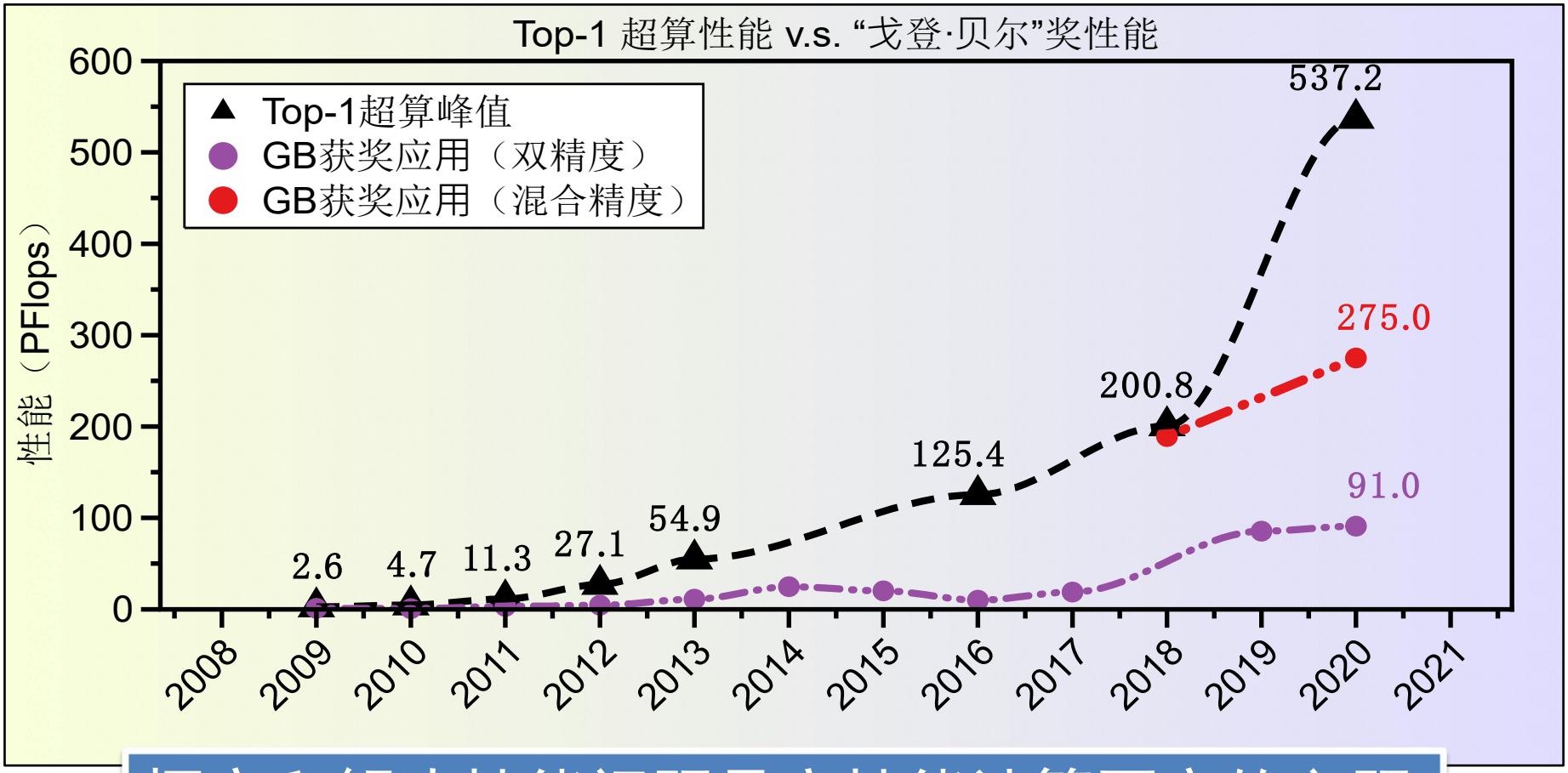



Table 1 Zettascale metrics

Metric	Value
Peak performance	1 Zflops
Power consumption	100 MW
Power efficiency	10 Tflops/W
Peak performance per node	10 Pflops/node
Bandwidth between nodes	1.6 Tb/s
I/O bandwidth	10–100 PB/s
Storage capacity	1 ZB
Floor space	1000 m^2

[Zettascale by 2035? China Thinks So (hpcwire.com)]
[Zettascale computing - Wikipedia]

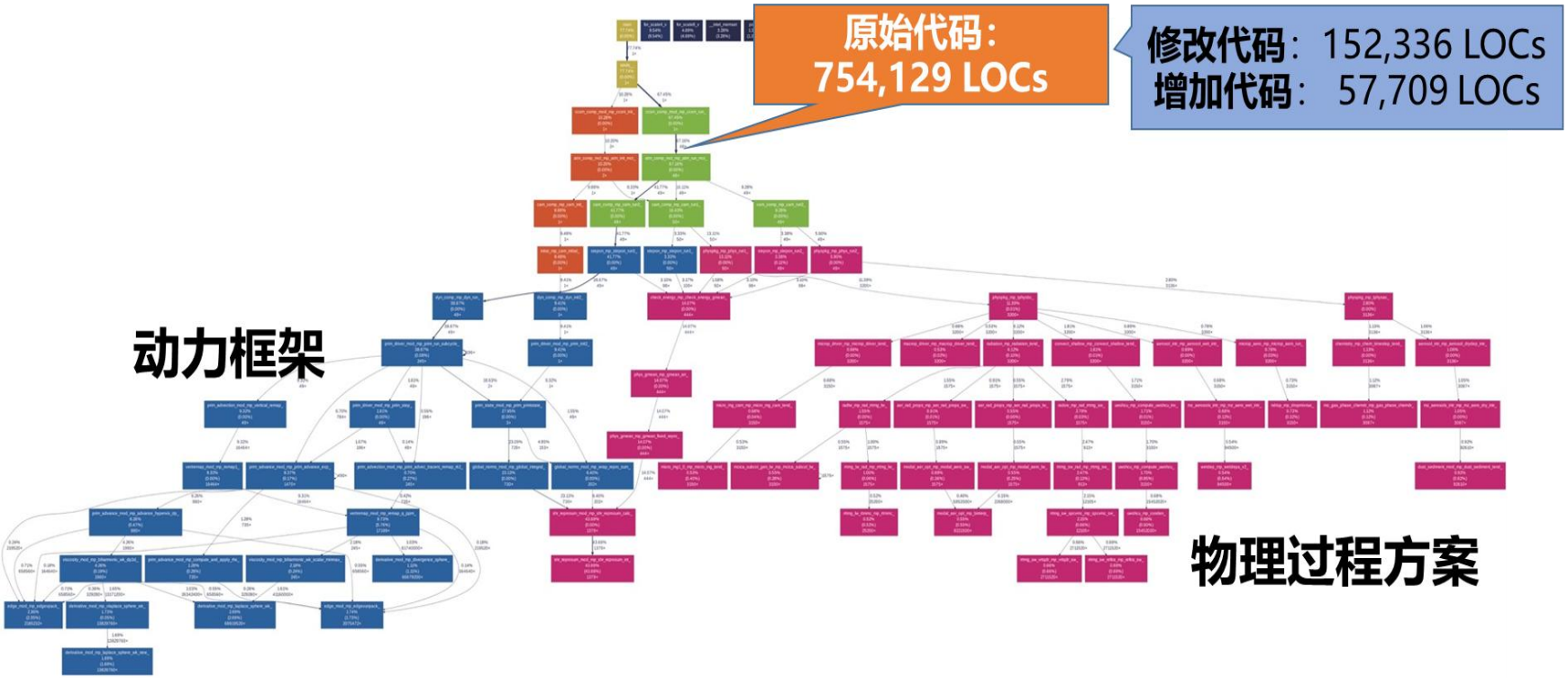
算力转化为科学工程难题求解能力挑战大



探究和解决性能问题是高性能计算不变的主题

超算平台的高移植与优化成本

- 社区大气模式CAM5在太湖之光上的移植和优化成本在10人年以上



性能可移植

提供统一并行编程模型，降低程序员开发复杂度

MPI | OpenMP | CUDA | UPC | AThread | OpenACC | C/Fortran



多样、复杂的体系结构

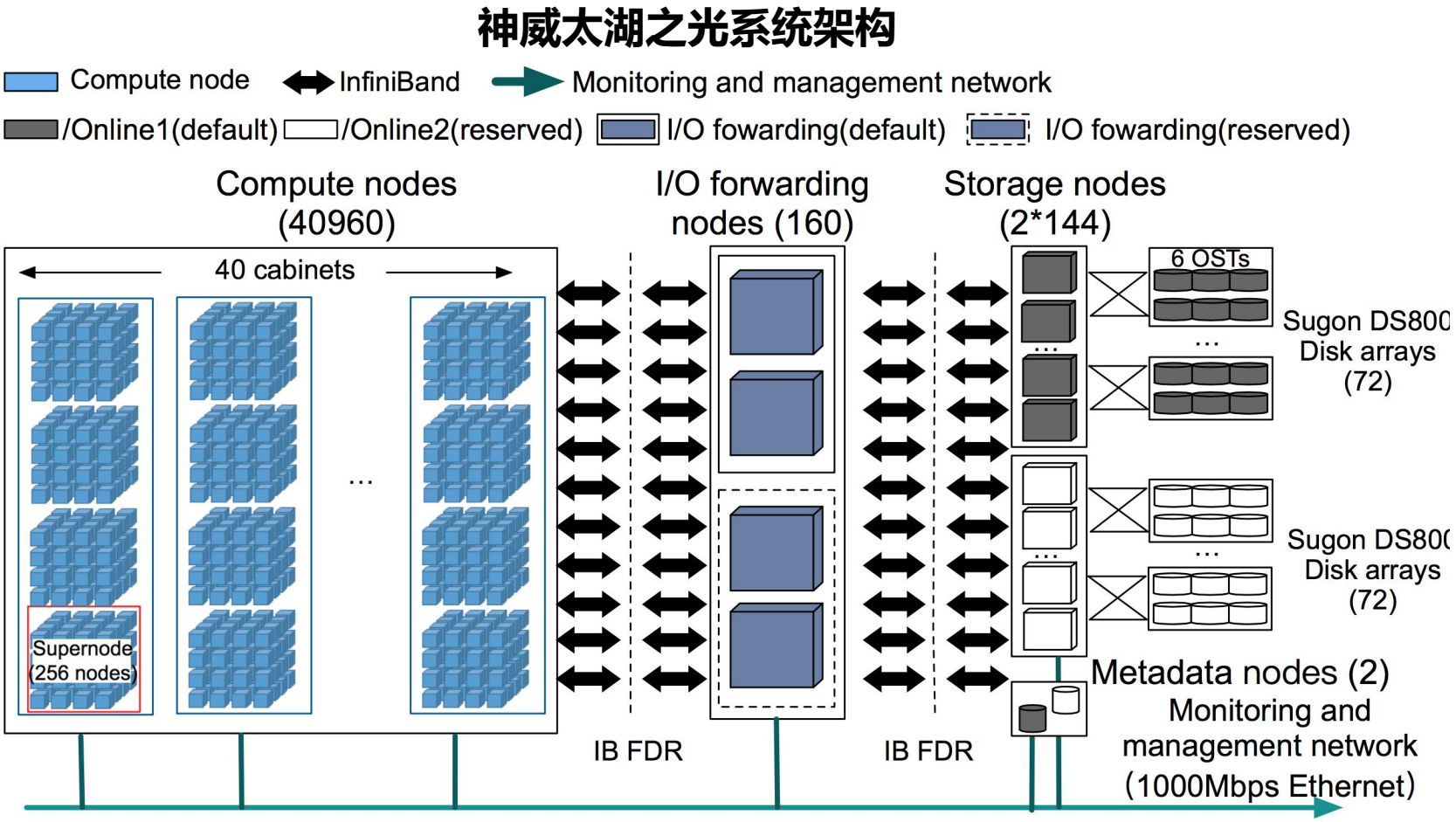


异构高性能计算机成为当前的主流趋势

一次编程，跨平台高效运行

- 设计面向领域特定并行编程模型
- 设计多层次中间表示
- 在不同层之间实现不同的优化选项
- 利用领域知识指导程序优化
- 针对不同高性能平台，挖掘底层硬件特征

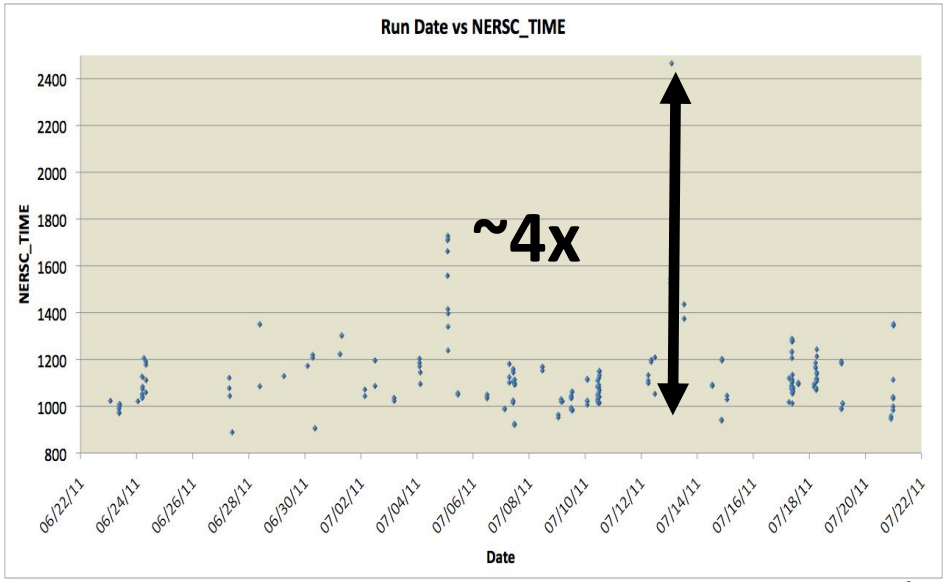
大规模计算系统的管理挑战



面对数万台设备，如何调度？如何容错？

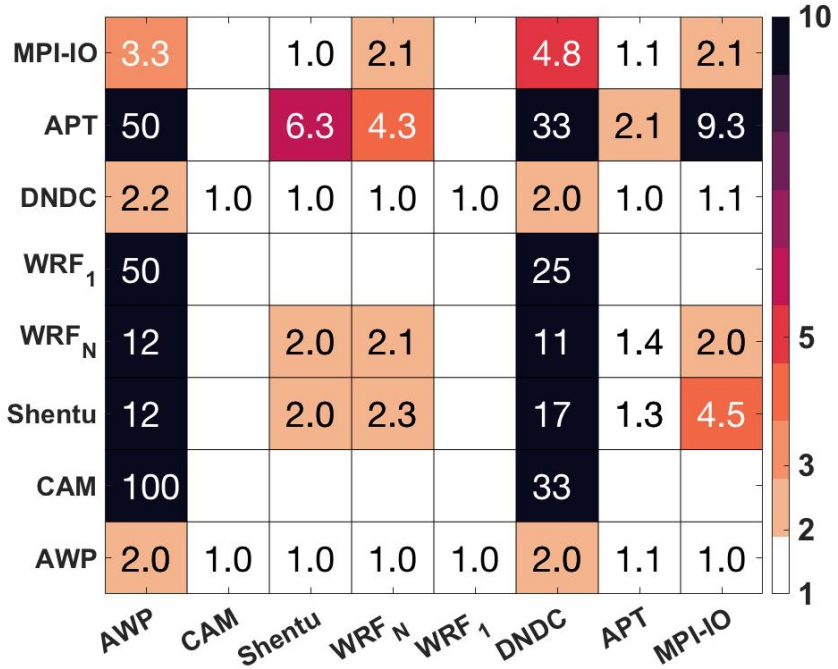
大规模计算系统的管理挑战

问题：同样的应用、配置和输入，在同一台超级计算系统上，多次运行性能不同，且难于重现



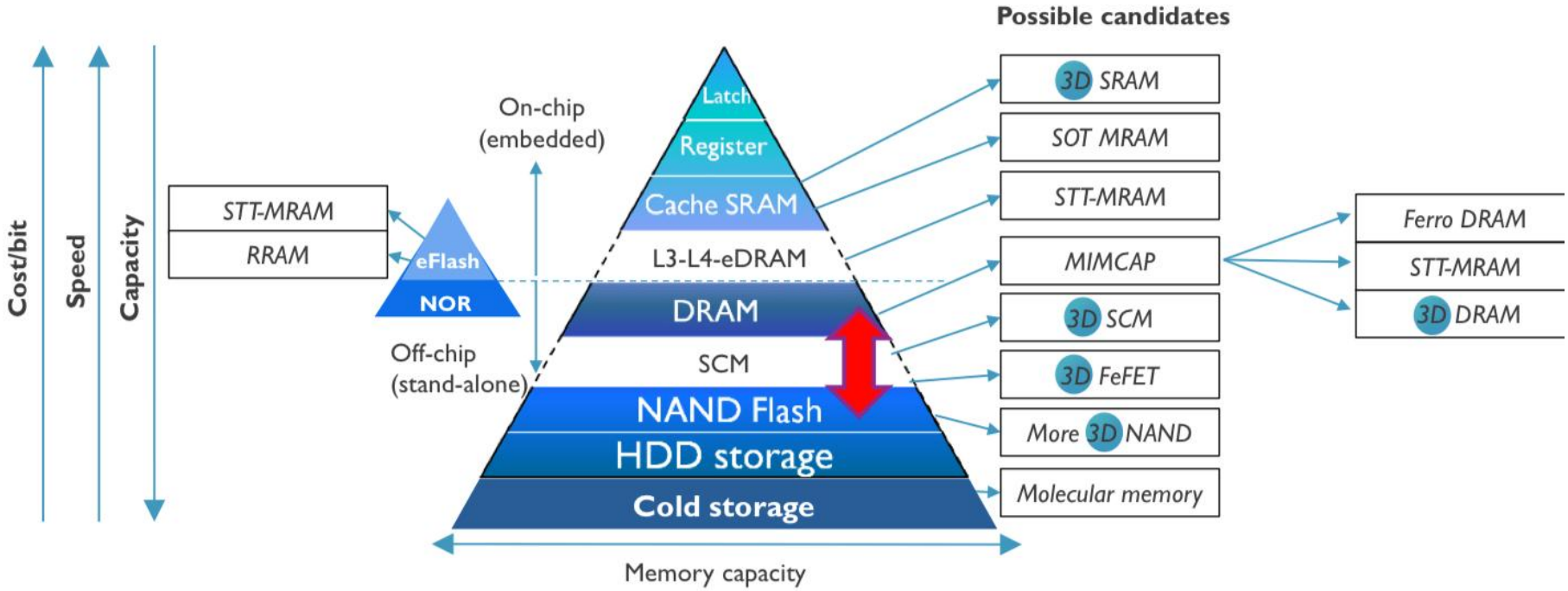
From ETH Prof. Torsten Hoefler

NERSC超算固定规模LINPACK测试



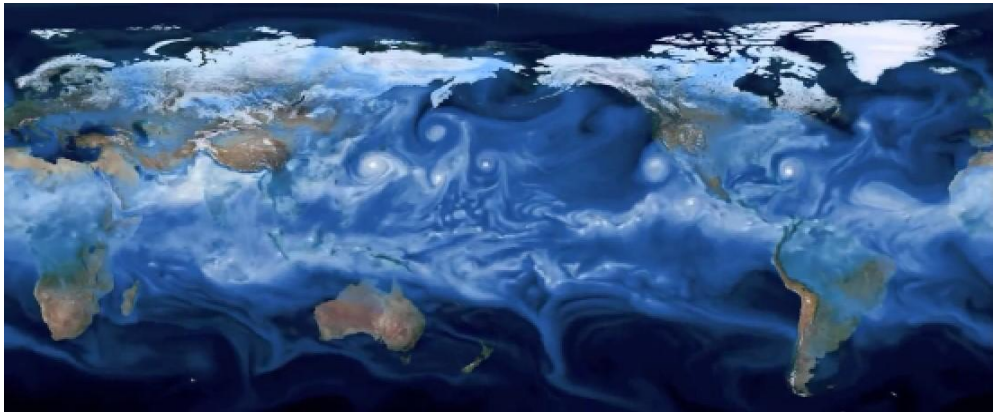
太湖之光应用IO性能测试

新硬件带来新挑战与新机遇

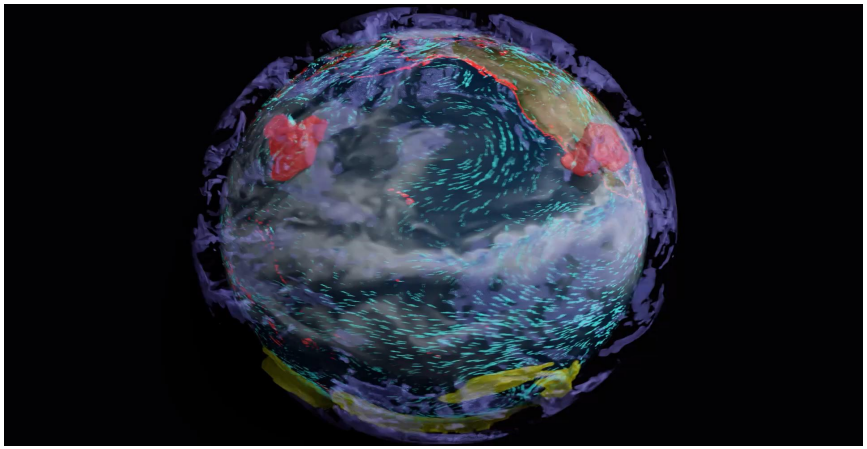


https://semiengineering.com/a-new-memory-contender/

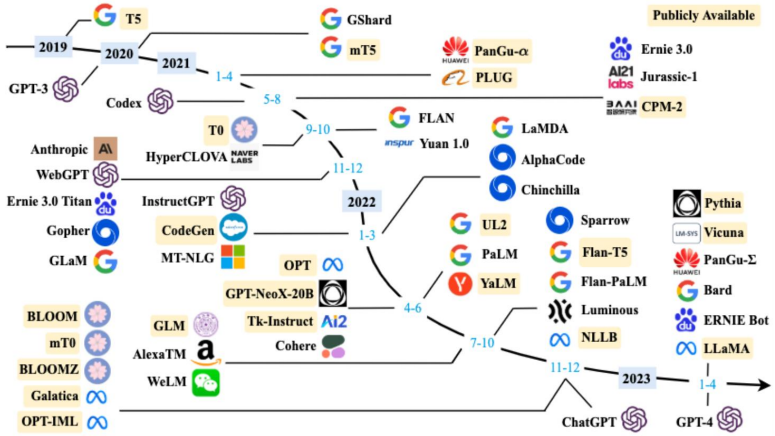
未来应用发展趋势：融合人工智能



劳伦斯伯克利国家实验室和NVIDIA公司的联合研究团队基于Summit利用高分辨率气候模拟数据集，有效识别和预测极端天气模式（2018年戈登贝尔奖）



NVIDIA to Build Earth-2 Supercomputer to See Our Future，to get million-x speedups
November 12, 2021 by JENSEN HUANG



训练大语言模型等大规模人工智能模型

总结

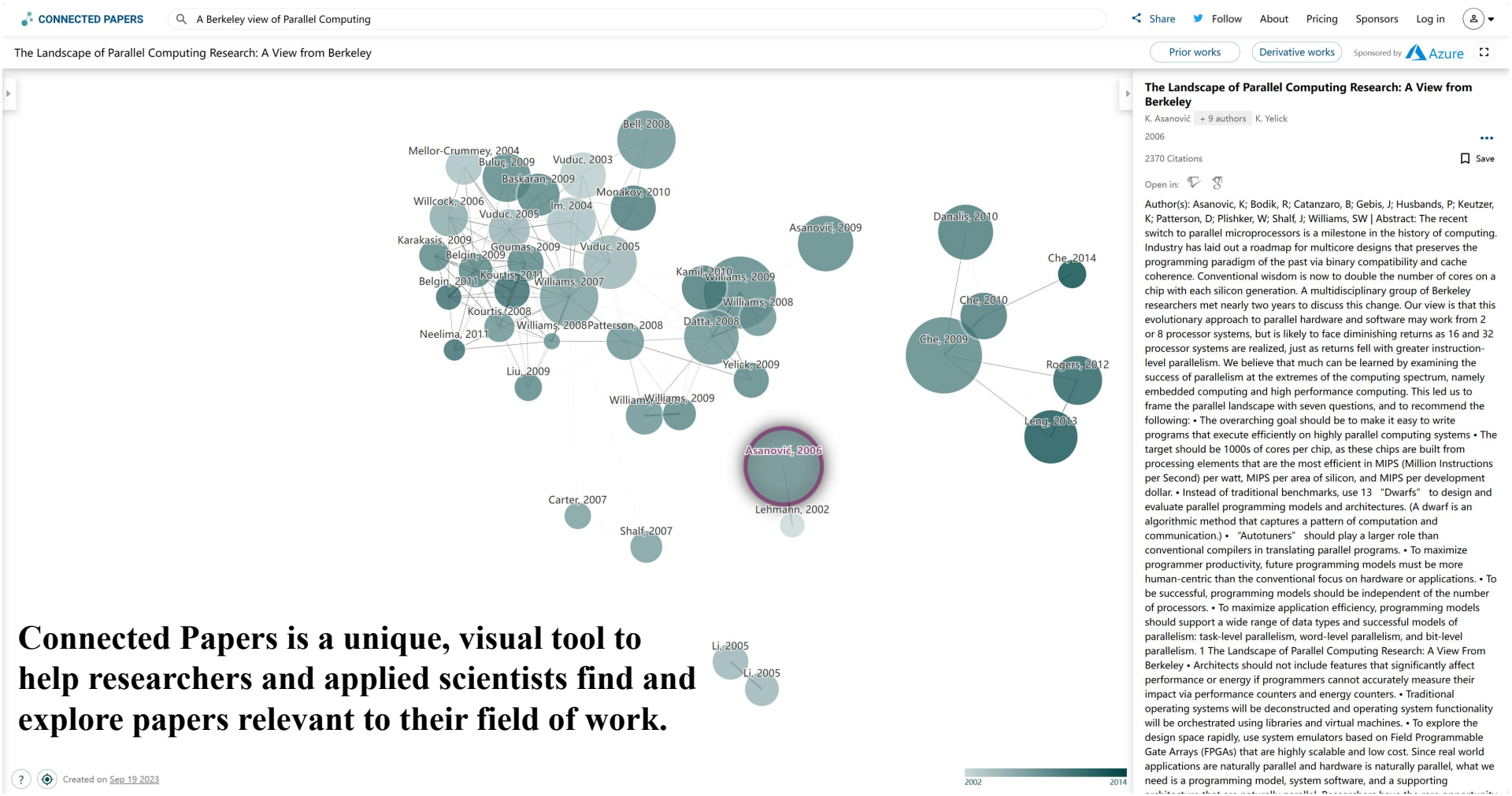
- 集群计算旨在解决有挑战性的计算和数据分析问题
  - 集群计算是高端计算平民化的代表
  - 应用需求推动着高性能计算机技术的快速发展
  - Z级计算机是高性能计算的下一个里程碑，也是各国竞争的焦点
  - Z级计算面临着诸多挑战，软硬件协同设计被认为是发展方向
- 随处理器技术发展，并行计算日趋普及
  - 并行处理是核心方法，正成为一种**通用计算技术**
  - 并行程序设计与原有串行程序设计相比更困难
  - 关键：理解并行思维、并行程序设计和体系结构，提升并行代码产率

- 如果想了解更多，请关注A Berkeley view of Parallel Computing, http://www.eecs.berkeley.edu/Pubs/TechRpts/2006/EECS-2006-183.pdf
- 相关知识和编程技术学习可以首先关注https://hpc.llnl.gov/training/tutorials

推荐阅读的文章和关注的网站

- A Berkeley view of Parallel Computing, http://www.eecs.berkeley.edu/Pubs/TechRpts/2006/EECS-2006-183.pdf
- There's plenty of room at the Top: What will drive computer performance after Moore's law? Science, 2020.6
- Demystifying Parallel and Distributed Deep Learning: An In-Depth Concurrency Analysis https://arxiv.org/abs/1802.09941
- 扩展阅读
  - www.top500.org
  - HPCwire: Global News on High Performance Computing (HPC)

Connected Papers



谢谢！

张武生、薛巍

清华大学计算机系

zws@tsinghua.edu.cn,
xuewei@tsinghua.edu.cn